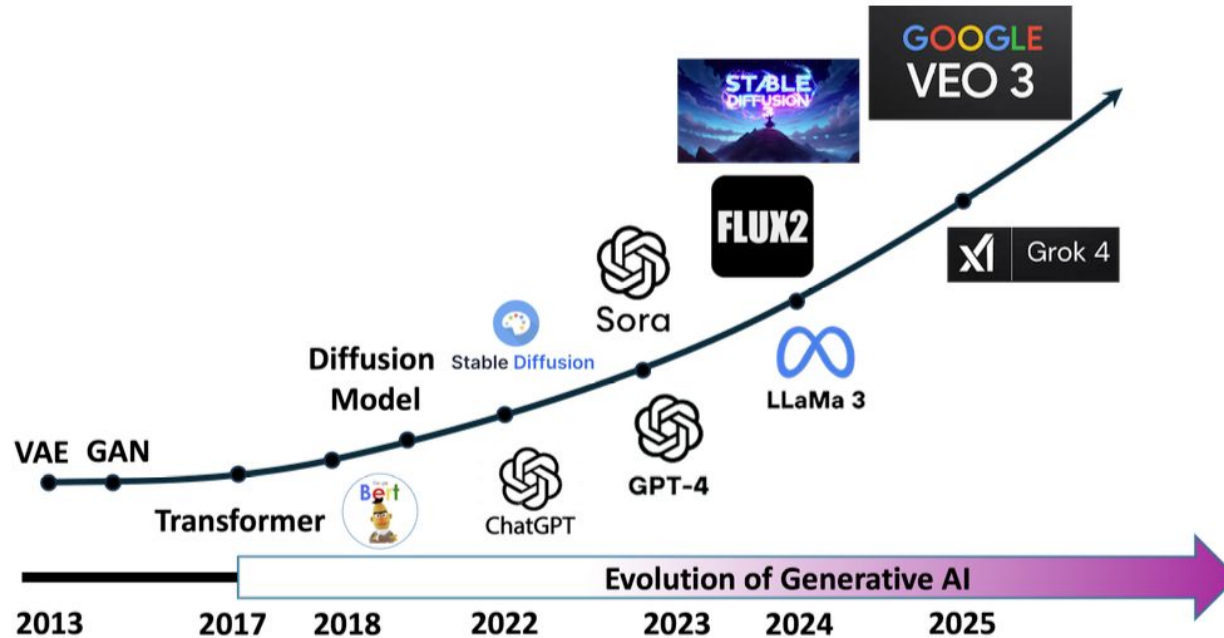


From Generative AI to Agentic AI

Kevin Murphy
August 2025

The generative AI revolution



Text to image generation

<https://deepmind.google/models/imagen/>



Prompt: Create a cinematic, photorealistic medium shot capturing the nostalgic warmth of a late 90s indie film. The focus is a young woman with brightly dyed pink hair...

Text to image generation

<https://deepmind.google/models/imagen/>

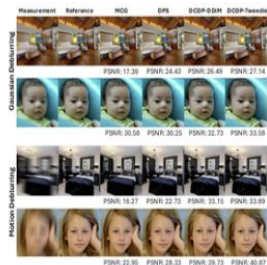


Prompt: Framed in a sophisticated medium shot, the white-throated kingfisher stretches its magnificent wings against the backdrop of a serene lake environment at perhaps dawn or dusk. The low angle of the warm light saturates the bird's plumage.

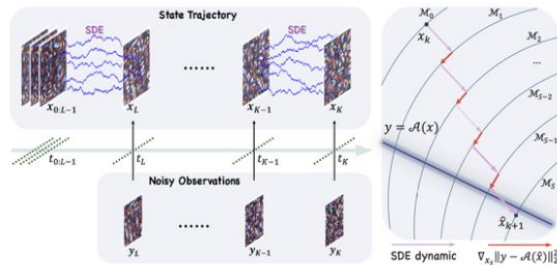
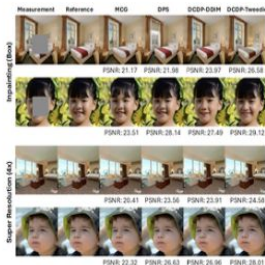
Text to video+audio generation
<https://deepmind.google/models/veo/>



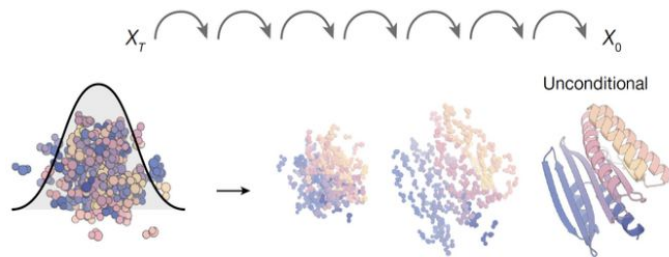
Scientific applications



Inverse Problems
(DCDP, Li et al.'24)

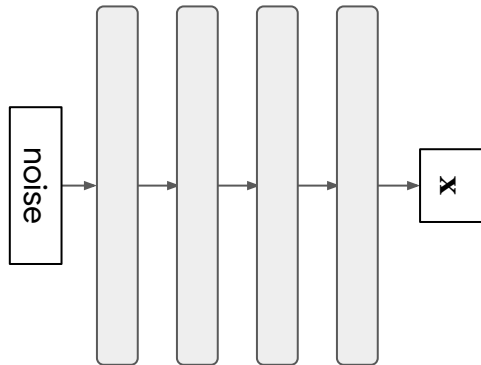
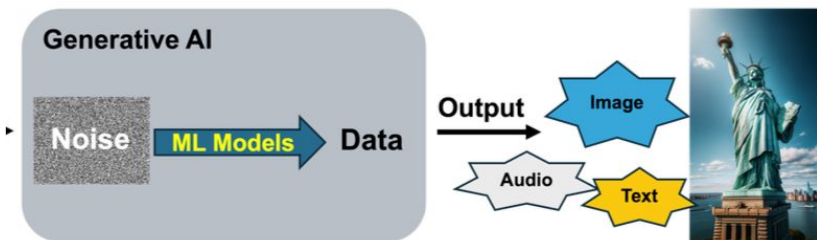


Data Assimilation
(FlowDAS, Chen et al.'25)



Protein Design
(RFDiffusion, Watson et al.'23)

Generative modelling: a probabilistic perspective



$$p(x) = \int p(x|z) p(z) dz$$

Conditional generative models

Control model output using a **control signal**:

$$p(x|c) \text{ vs. } p(x)$$

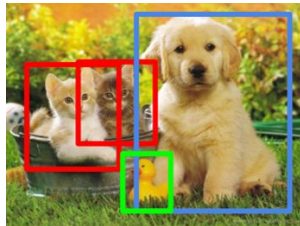
sparsely conditioned

densely conditioned

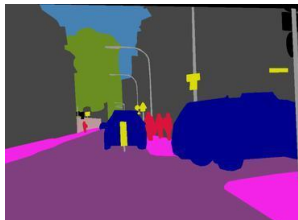
class labels

$y = \text{"cat"}$

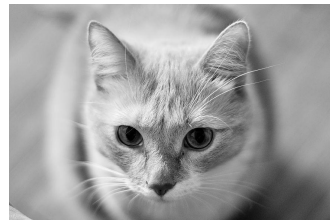
bounding boxes



segmentation

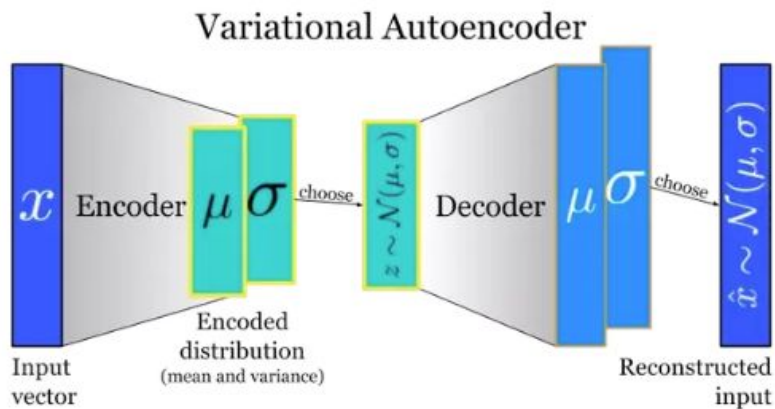


grayscale image
(colorisation)

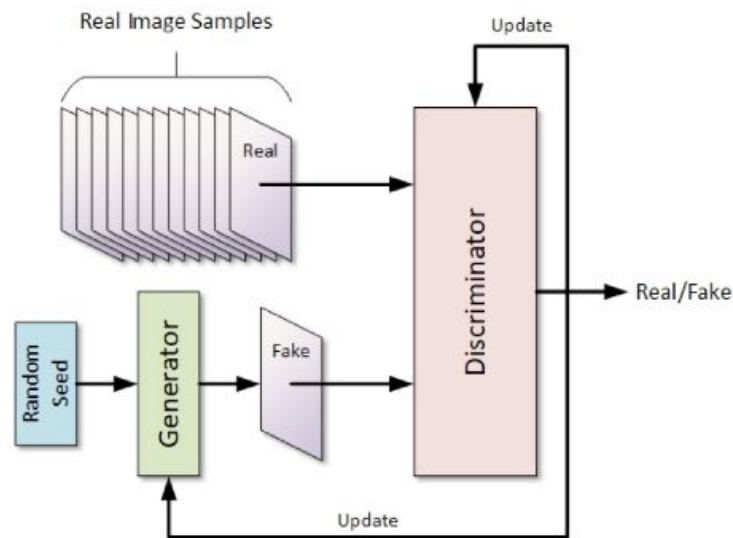


Previous approaches: one-shot generation

Generative models in the past:



(a) VAE (Kingma & Welling, 2013):
poor generation quality.

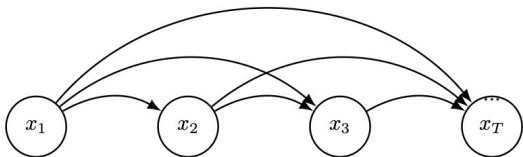


(b) GAN (Goodfellow et al. 2014):
unstable to train on large datasets.

Modern approaches: iterative (sequential) generation

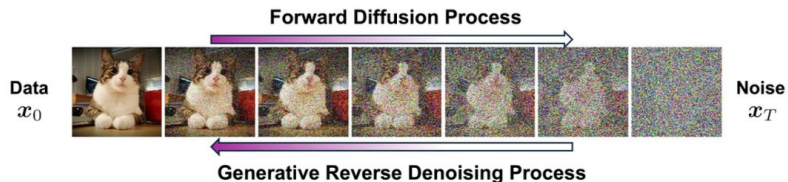
Autoregression: step-by-step

Turn everything into a 1D sequence,
generate it one step at a time



Diffusion: iterative denoising

Gradually add noise until all information is destroyed,
then learn to invert this procedure step by step



This talk: Speeding up the generative process

Continuous Diffusion

Gradually add noise until all information is destroyed,
then learn to invert this procedure step by step

ICML 2025

Distributional Diffusion Models with Scoring Rules

Valentin De Bortoli, Alexandre Galashov, J. Swaroop Guntupalli, Guangyao Zhou, Kevin Murphy, Arthur Gretton, Arnaud Doucet

Discrete Diffusion: parallel token generation

ICML 2023

Muse: Text-To-Image Generation via Masked Generative Transformers

Huiwen Chang, Han Zhang, Jarred Barber, AJ Maschinot, Jose Lezama, Lu Jiang, Ming-Hsuan Yang, Kevin Murphy, William Freeman, Michael Rubinstein, Yanzhen Li, Dilip Krishnan

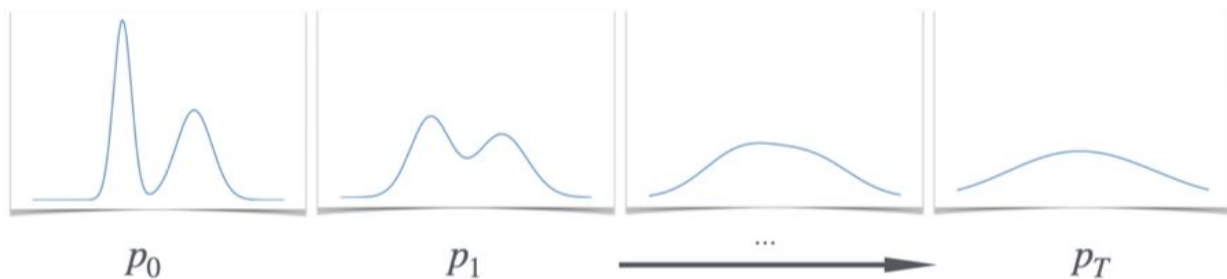
Speeding up (continuous) diffusion

ICML 2025

Distributional Diffusion Models with Scoring Rules

Valentin De Bortoli, Alexandre Galashov, J. Swaroop Guntupalli, Guangyao Zhou, Kevin Murphy, Arthur Gretton, Arnaud Doucet

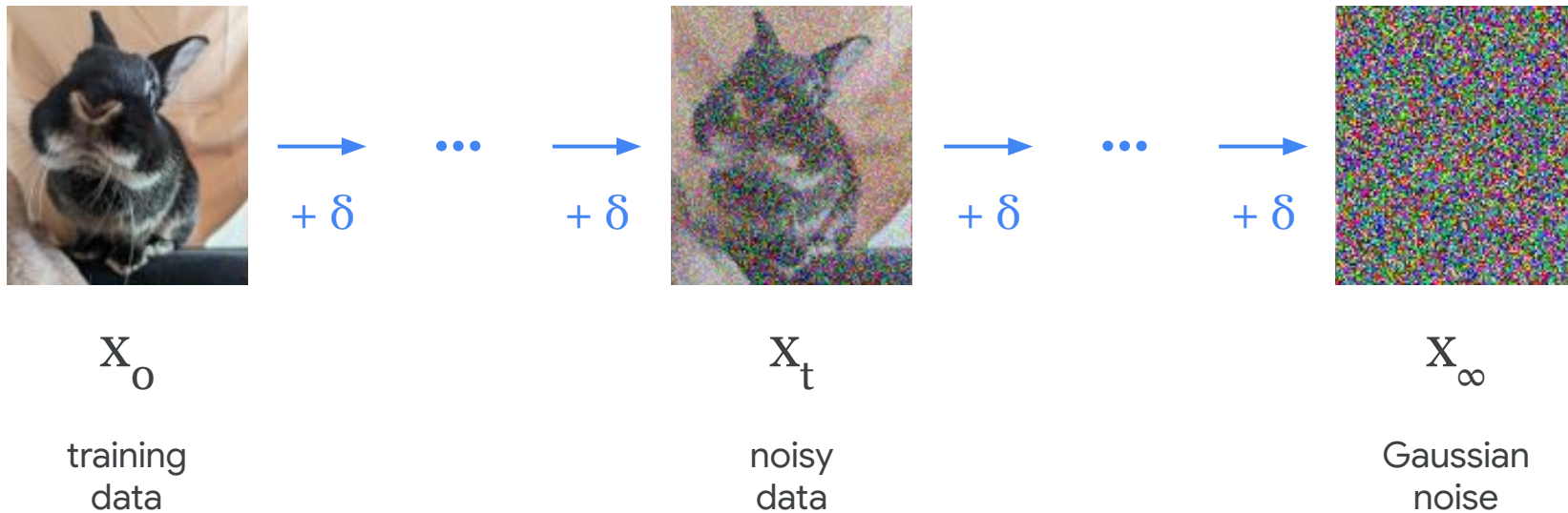
Diffusion: mapping from data distribution to Gaussian (and back!)



[Video](#) from Welch Labs / 3Blue 1 Brown

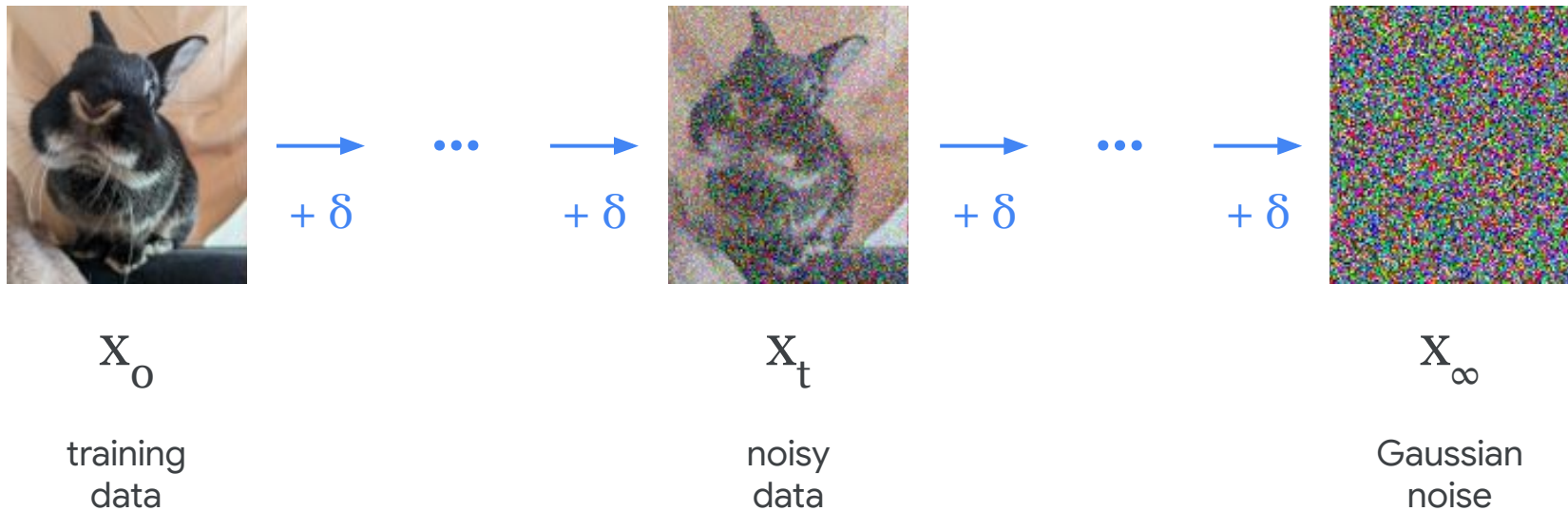
S. H. Chan, "[Tutorial on Diffusion Models for Imaging and Vision](#)," *Found. Trends® Comput. Graph. Vis.*, Mar. 2024,
Nakkiran et al, "[Step-by-Step Diffusion: An Elementary Tutorial](#)", arxiv, 2024

Diffusion: forward process



$$\delta \sim \mathcal{N}(0, \sigma^2)$$

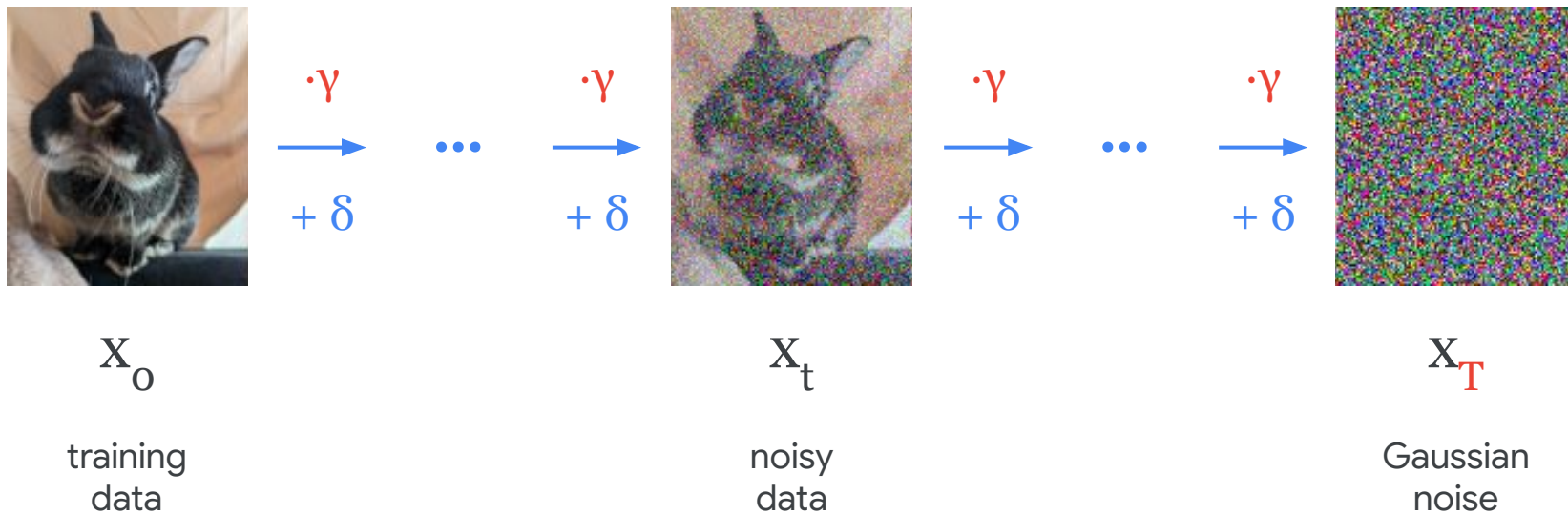
Diffusion: forward process



$$X_t = X_0 + \sigma(t) \cdot \varepsilon$$

$$\varepsilon \sim \mathcal{N}(0, 1)$$

Diffusion: forward process



$$x_t = \alpha(t) \cdot x_0 + \sigma(t) \cdot \varepsilon$$

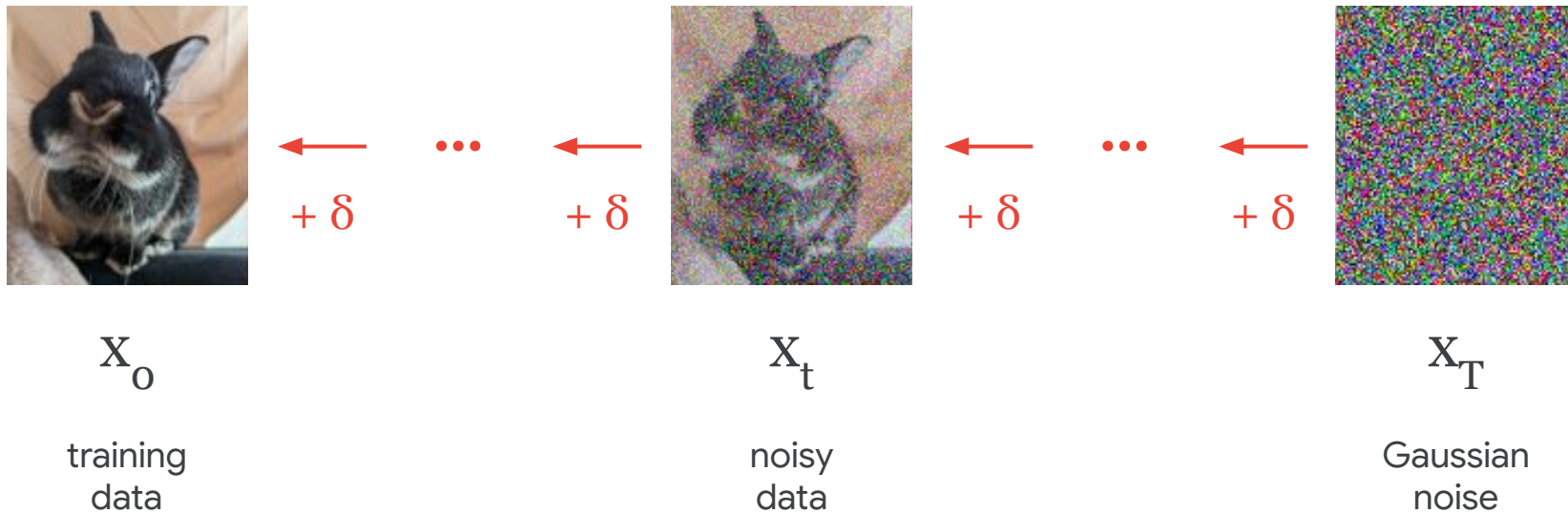
$$\varepsilon \sim \mathcal{N}(0, 1)$$

Diffusion: forward process

- $\sigma(t)$ is the **noise schedule**
- controls the **rate of corruption** over the course of the process
- Several choices for $\alpha(t)$:
 - variance-preserving (VP)
 $\alpha = \sqrt{1 - \sigma^2}$
 - variance-exploding (VE)
 $\alpha = 1$
 - rectified flow, flow matching (RF)
 $\alpha = 1 - \sigma$



Diffusion: backward process



$$\delta = ???$$

Reverse process is approximately Gaussian

Claim 1 (Informal). *Let $p_{t-\Delta t}(x)$ be an arbitrary, sufficiently-smooth density over $\mathbb{R}^d$. Consider the joint distribution of $(x_{t-\Delta t}, x_t)$, where $x_{t-\Delta t} \sim p_{t-\Delta t}$ and $x_t \sim x_{t-\Delta t} + \mathcal{N}(0, \sigma_q^2 \Delta t)$. Then, for sufficiently small Δt , the following holds. For all conditionings $z \in \mathbb{R}^d$, there exists μ_z such that:*

$$p(x_{t-\Delta t} \mid x_t = z) \approx \mathcal{N}(x_{t-\Delta t}; \mu_z, \sigma_q^2 \Delta t). \quad (16)$$

for some constant μ_z depending only on z . Moreover, it suffices to take¹⁵

$$\mu_z := \mathbb{E}_{(x_{t-\Delta t}, x_t)} [x_{t-\Delta t} \mid x_t = z] \quad (17)$$

Proof by Bayes rule and (first order) Taylor approximation

$$p(x_{t-\Delta t}|x_t) = p(x_t|x_{t-\Delta t})p_{t-\Delta t}(x_{t-\Delta t})/p_t(x_t)$$

$$\log p(x_{t-\Delta t}|x_t) = \log p(x_t|x_{t-\Delta t}) + \log p_{t-\Delta t}(x_{t-\Delta t}) - \log p_t(x_t)$$

$$= \log p(x_t|x_{t-\Delta t}) + \log p_t(x_{t-\Delta t}) + \mathcal{O}(\Delta t)$$

$$= -\frac{1}{2\sigma_q^2\Delta t} \|x_{t-\Delta t} - x_t\|_2^2 + \log p_t(x_{t-\Delta t})$$

$$= -\frac{1}{2\sigma_q^2\Delta t} \|x_{t-\Delta t} - x_t\|_2^2 + \log p_t(x_t) + \langle \nabla_x \log p_t(x_t), (x_{t-\Delta t} - x_t) \rangle + \mathcal{O}(\Delta t)$$

$$= -\frac{1}{2\sigma_q^2\Delta t} \left(\|x_{t-\Delta t} - x_t\|_2^2 - 2\sigma_q^2\Delta t \langle \nabla_x \log p_t(x_t), (x_{t-\Delta t} - x_t) \rangle \right)$$

$$= -\frac{1}{2\sigma_q^2\Delta t} \|x_{t-\Delta t} - x_t - \sigma_q^2\Delta t \nabla_x \log p_t(x_t)\|_2^2 + C$$

$$= -\frac{1}{2\sigma_q^2\Delta t} \|x_{t-\Delta t} - \mu\|_2^2$$

Drop constants involving only x_t .

Since $p_{t-\Delta t}(\cdot) = p_t(\cdot) + \Delta t \frac{\partial}{\partial t} p_t(\cdot)$.

Definition of $\log p(x_t|x_{t-\Delta t})$.

Taylor expand around x_t and drop constants.

Complete the square in $(x_{t-\Delta t} - x_t)$, and drop constant C involving only x_t .

For $\mu := x_t + (\sigma_q^2\Delta t)\nabla_x \log p_t(x_t)$.

This is identical, up to additive factors, to the log-density of a Normal distribution with mean μ and variance $\sigma_q^2\Delta t$. Therefore,

$$p(x_{t-\Delta t} | x_t) \approx \mathcal{N}(x_{t-\Delta t}; \mu, \sigma_q^2\Delta t). \quad (20)$$

Training the model to predict mean of next step: $\mathbb{E}[X_{t-1} | x_t]$

$\mu_t(z) := \mathbb{E}[x_t | x_{t+\Delta t} = z]$. Neural network $f_\theta(z, t)$ eg U-net, transformer

$\mu_t = \operatorname{argmin}_{f: \mathbb{R}^d \rightarrow \mathbb{R}^d} \mathbb{E}_{x_t, x_{t+\Delta t}} \|f(x_{t+\Delta t}) - x_t\|_2^2$, (Nonlinear) least squares

Pseudocode 1: DDPM train loss

Input: Neural network f_θ ; Sample-access to target distribution p .

Data: Terminal variance σ_q ; step-size Δt .

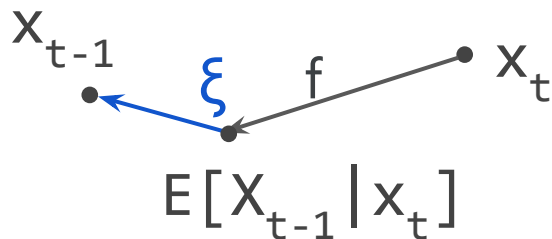
Output: Stochastic loss L_θ

```
1  $x_0 \leftarrow \text{Sample}(p)$ 
2  $t \leftarrow \text{Unif}[0, 1]$ 
3  $x_t \leftarrow x_0 + \mathcal{N}(0, \sigma_q^2 t)$ 
4  $x_{t+\Delta t} \leftarrow x_t + \mathcal{N}(0, \sigma_q^2 \Delta t)$ 
5  $L \leftarrow \|f_\theta(x_{t+\Delta t}, t + \Delta t) - x_t\|_2^2$ 
6 return  $L_\theta$ 
```

"Denoising Diffusion Probabilistic Models (DDPM)", Jonathan Ho, Ajay Jain, and Pieter Abbeel, NIPS 2020

Source: "[Step-by-Step Diffusion: An Elementary Tutorial](#)", Nakkiran et al, 2024

Inference (generation): 1/3



Pseudocode 2: DDPM sampling (Code for Algorithm 1)

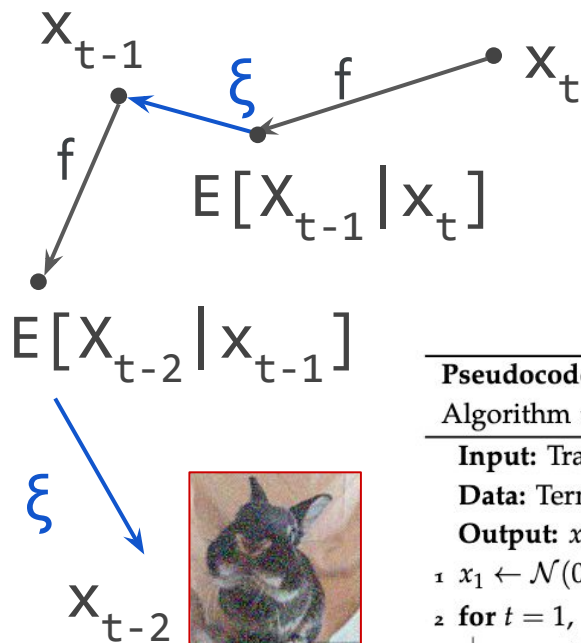
Input: Trained model f_θ .

Data: Terminal variance σ_q ; step-size Δt .

Output: x_0

```
1  $x_1 \leftarrow \mathcal{N}(0, \sigma_q^2)$ 
2 for  $t = 1, (1 - \Delta t), (1 - 2\Delta t), \dots, \Delta t$  do
3    $\eta \leftarrow \mathcal{N}(0, \sigma_q^2 \Delta t)$ 
4    $x_{t-\Delta t} \leftarrow f_\theta(x_t, t) + \eta$ 
5 end
6 return  $x_0$ 
```

Inference (generation): 2/3



Pseudocode 2: DDPM sampling (Code for Algorithm 1)

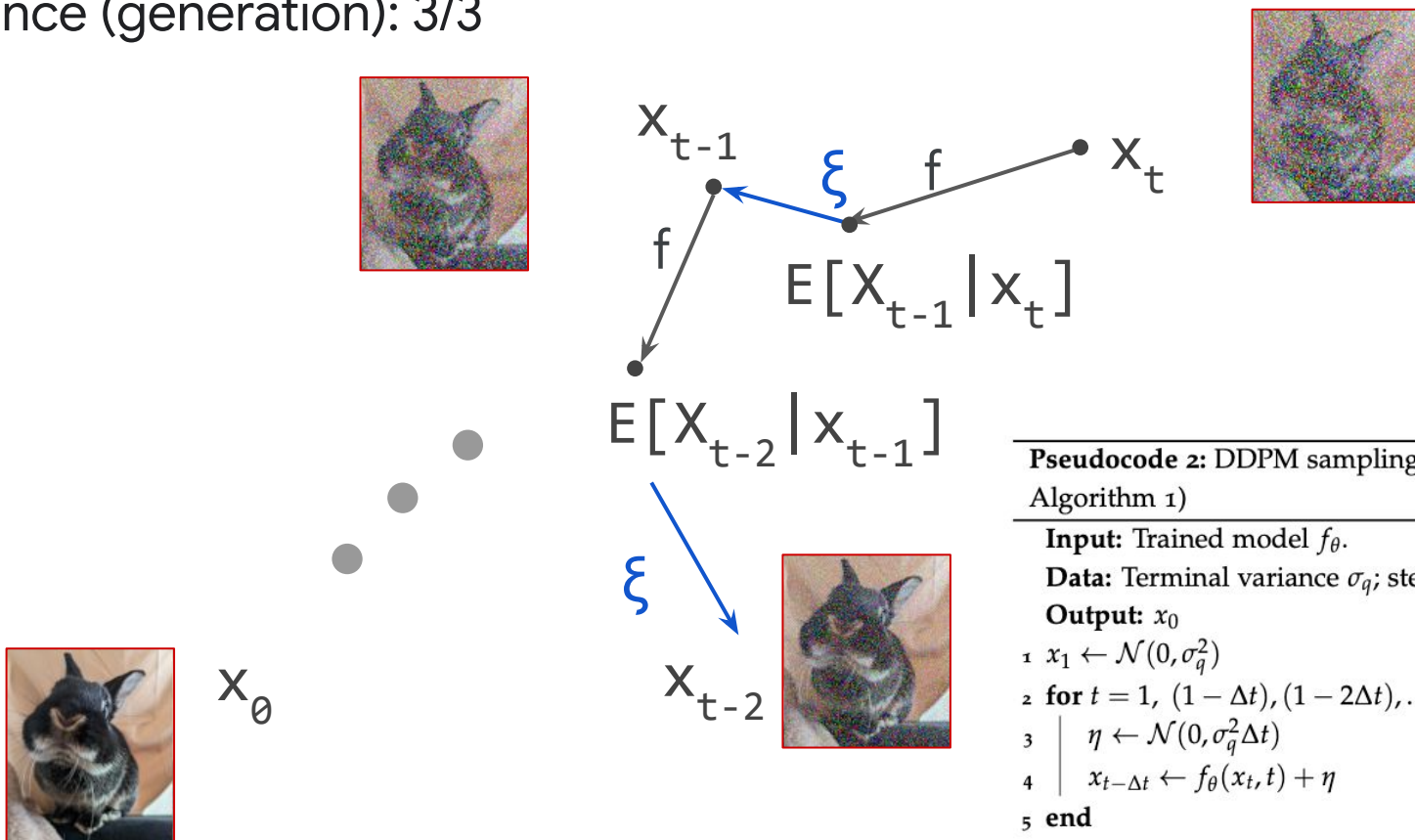
Input: Trained model f_θ .

Data: Terminal variance σ_q ; step-size Δt .

Output: x_0

```
1  $x_1 \leftarrow \mathcal{N}(0, \sigma_q^2)$ 
2 for  $t = 1, (1 - \Delta t), (1 - 2\Delta t), \dots, \Delta t$  do
3    $\eta \leftarrow \mathcal{N}(0, \sigma_q^2 \Delta t)$ 
4    $x_{t-\Delta t} \leftarrow f_\theta(x_t, t) + \eta$ 
5 end
6 return  $x_0$ 
```

Inference (generation): 3/3



Pseudocode 2: DDPM sampling (Code for Algorithm 1)

Input: Trained model f_θ .

Data: Terminal variance σ_q ; step-size Δt .

Output: x_0

```

1  $x_1 \leftarrow \mathcal{N}(0, \sigma_q^2)$ 
2 for  $t = 1, (1 - \Delta t), (1 - 2\Delta t), \dots, \Delta t$  do
3    $\eta \leftarrow \mathcal{N}(0, \sigma_q^2 \Delta t)$ 
4    $x_{t-\Delta t} \leftarrow f_\theta(x_t, t) + \eta$ 
5 end
6 return  $x_0$ 
```

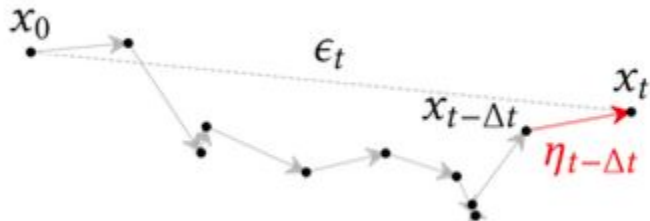
Training the model to predict mean of target $\mathbb{E}[X_\theta | x_t]$

Claim 2. For the Gaussian diffusion setting of Section 1.3, we have:

$$\mathbb{E}[(x_{t-\Delta t} - x_t) | x_t] = \frac{\Delta t}{t} \mathbb{E}[(x_0 - x_t) | x_t].$$

Or equivalently:

$$\mathbb{E}[x_{t-\Delta t} | x_t] = \left(\frac{\Delta t}{t}\right) \mathbb{E}[x_0 | x_t] + \left(1 - \frac{\Delta t}{t}\right) x_t.$$



Predicting fully denoised target x_0 is a lower variance loss than predicting one-step denoised target x_{t-1}

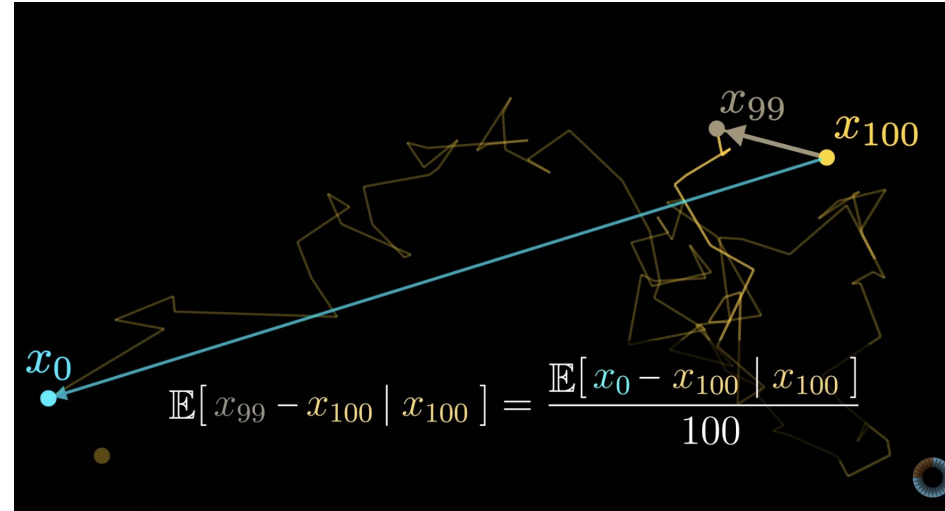
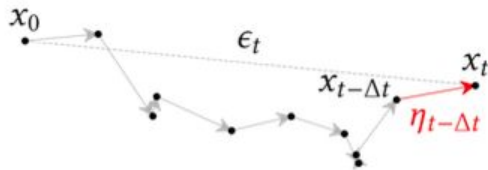
Training the model to predict mean of target $E[X_\theta | x_t]$

Claim 2. For the Gaussian diffusion setting of Section 1.3, we have:

$$\mathbb{E}[(x_{t-\Delta t} - x_t) | x_t] = \frac{\Delta t}{t} \mathbb{E}[(x_0 - x_t) | x_t].$$

Or equivalently:

$$\mathbb{E}[x_{t-\Delta t} | x_t] = \left(\frac{\Delta t}{t}\right) \mathbb{E}[x_0 | x_t] + \left(1 - \frac{\Delta t}{t}\right) x_t.$$

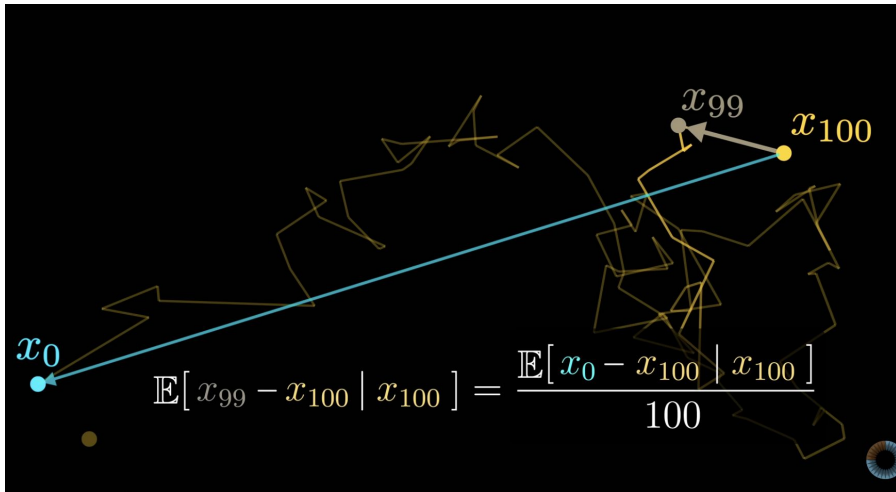


Predicting fully denoised target x_0 is a lower variance loss than predicting one-step denoised target x_{t-1}

Training the model to predict mean of target $\mathbb{E}[X_\theta | x_t]$

$$g_\theta(x, t) \approx \mathbb{E}[x_0 | x_t]$$

$$\mathbb{E}[(x_{t-\Delta t} - x_t) | x_t] = \frac{\Delta t}{t} \mathbb{E}[(x_0 - x_t) | x_t].$$



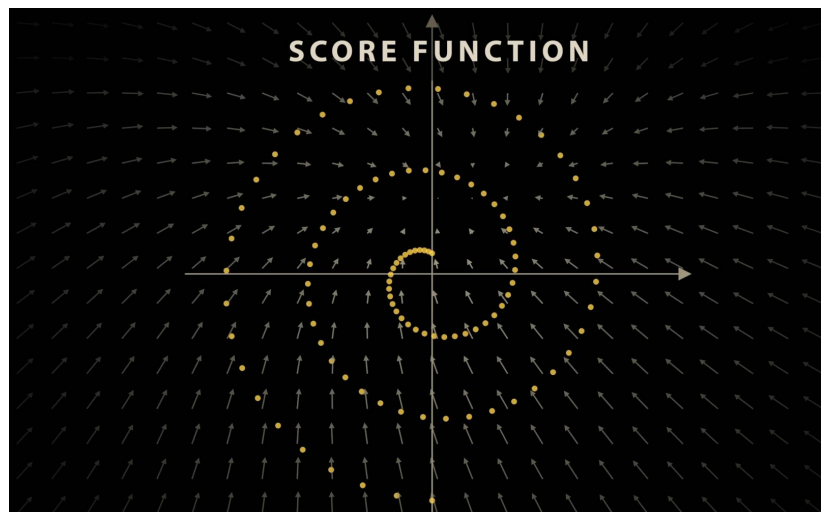
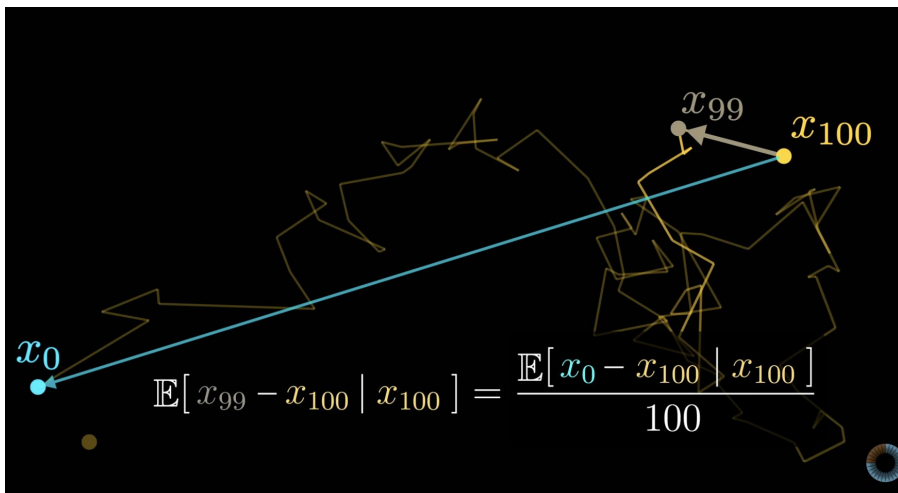
Training the model to predict score function

$$g_{\theta}(x, t) \approx \mathbb{E}[x_0 \mid x_t]$$

$$\mathbb{E}[(x_{t-\Delta t} - x_t) \mid x_t] = \frac{\Delta t}{t} \mathbb{E}[(x_0 - x_t) \mid x_t].$$

$$\mu_z := \mathbb{E}_{(x_{t-\Delta t}, x_t)} [x_{t-\Delta t} \mid x_t = z]$$

$$= z + (\sigma_q^2 \Delta t) \nabla \log p_t(z),$$



Training the model to predict mean of target $\mathbb{E}[X_\theta | x_t]$

$$g_\theta(x, t) \approx \mathbb{E}[x_0 | x_t]$$

$$\begin{aligned}\mu_z &:= \mathbb{E}_{(x_{t-\Delta t}, x_t)} [x_{t-\Delta t} | x_t = z] \\ &= z + (\sigma_q^2 \Delta t) \nabla \log p_t(z),\end{aligned}$$

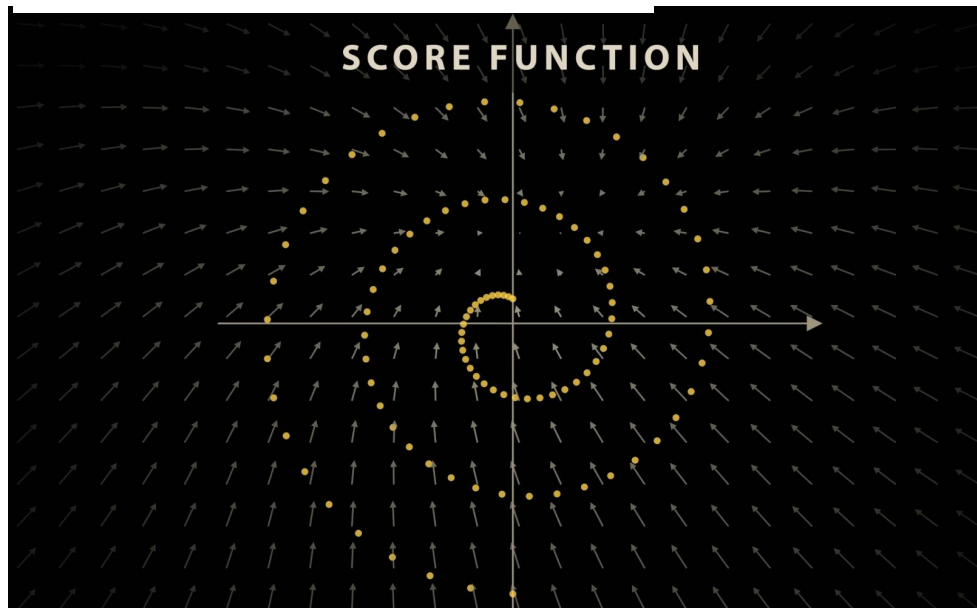
Pseudocode 6: DDPM train loss (x_0 -prediction)

Input: Neural network g_θ ; Sample-access to train distribution p .

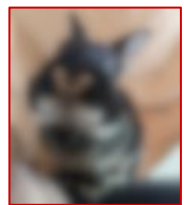
Data: Terminal variance σ_q

Output: Stochastic loss L

```
1  $x_0 \leftarrow \text{Sample}(p)$ 
2  $t \leftarrow \text{Unif}[0, 1]$ 
3  $x_t \leftarrow x_0 + \mathcal{N}(0, \sigma_q^2 t)$ 
4  $L \leftarrow \|g_\theta(x_t, t) - x_0\|_2^2$ 
5 return  $L$ 
```



Inference (generation): 1/3



$\hat{x}_{0|t}$

$$\eta \leftarrow \mathcal{N}(0, \sigma_q^2 \Delta t)$$

$$x_{t-\Delta t} \leftarrow f_\theta(x_t, t) + \eta$$

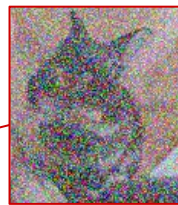
$$f_\theta(x, t) = (\Delta t/t)g_\theta(x, t) + (1 - \Delta t/t)x.$$

x_{t-1}
 $E[x_{t-1} | x_t]$



ξ

x_t



Pseudocode 7: DDPM sampling (x_0 -prediction)

Input: Trained model f_θ .

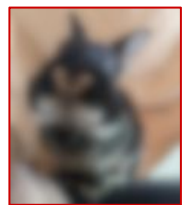
Data: Terminal variance σ_q ; step-size Δt .

Output: x_0

```

1  $x_1 \leftarrow \mathcal{N}(0, \sigma_q^2)$ 
2 for  $t = 1, (1 - \Delta t), (1 - 2\Delta t), \dots, \Delta t$  do
3    $\hat{\eta}_t \leftarrow g_\theta(x_t, t) - x_t$ 
4    $x_{t-\Delta t} \leftarrow x_t + (1/t)\hat{\eta}_t\Delta t + \mathcal{N}(0, \sigma_q^2\Delta t)$ 
5 end
6 return  $x_0$ 
```

Inference (generation)



$\hat{x}_{0|t}$

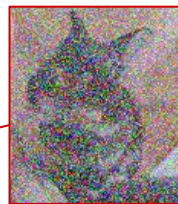
g

x_{t-1}



ξ

x_t



Pseudocode 7: DDPM sampling (x_0 -prediction)

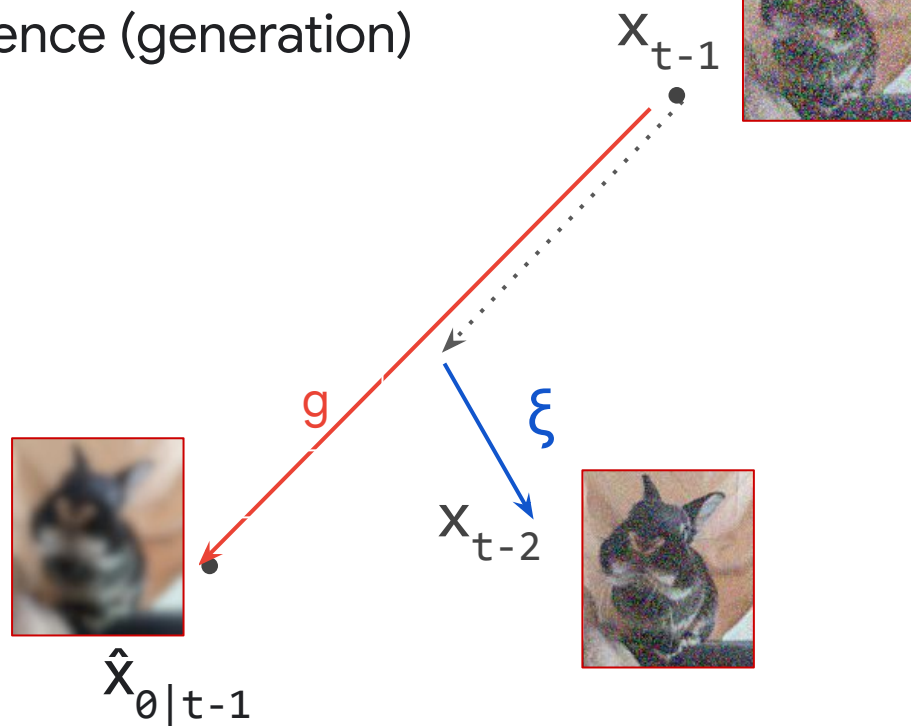
Input: Trained model f_θ .

Data: Terminal variance σ_q ; step-size Δt .

Output: x_0

```
1  $x_1 \leftarrow \mathcal{N}(0, \sigma_q^2)$ 
2 for  $t = 1, (1 - \Delta t), (1 - 2\Delta t), \dots, \Delta t$  do
3    $\hat{\eta}_t \leftarrow g_\theta(x_t, t) - x_t$ 
4    $x_{t-\Delta t} \leftarrow x_t + (1/t)\hat{\eta}_t\Delta t + \mathcal{N}(0, \sigma_q^2\Delta t)$ 
5 end
6 return  $x_0$ 
```

Inference (generation)



x_t



Pseudocode 7: DDPM sampling (x_0 -prediction)

Input: Trained model f_θ .

Data: Terminal variance σ_q ; step-size Δt .

Output: x_0

```
1  $x_1 \leftarrow \mathcal{N}(0, \sigma_q^2)$ 
2 for  $t = 1, (1 - \Delta t), (1 - 2\Delta t), \dots, \Delta t$  do
3    $\hat{\eta}_t \leftarrow g_\theta(x_t, t) - x_t$ 
4    $x_{t-\Delta t} \leftarrow x_t + (1/t)\hat{\eta}_t\Delta t + \mathcal{N}(0, \sigma_q^2\Delta t)$ 
5 end
6 return  $x_0$ 
```

Inference (generation): 3/3

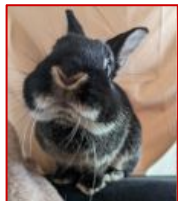
x_{t-1}



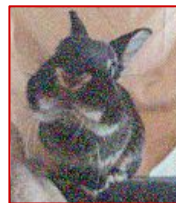
x_t



x_0



x_{t-2}



Pseudocode 7: DDPM sampling (x_0 -prediction)

Input: Trained model f_θ .

Data: Terminal variance σ_q ; step-size Δt .

Output: x_0

```
1  $x_1 \leftarrow \mathcal{N}(0, \sigma_q^2)$ 
2 for  $t = 1, (1 - \Delta t), (1 - 2\Delta t), \dots, \Delta t$  do
3    $\hat{\eta}_t \leftarrow g_\theta(x_t, t) - x_t$ 
4    $x_{t-\Delta t} \leftarrow x_t + (1/t)\hat{\eta}_t\Delta t + \mathcal{N}(0, \sigma_q^2\Delta t)$ 
5 end
6 return  $x_0$ 
```

Training the model to predict mean of the noise: $E[\epsilon_t | x_t]$

$$x_t = x_0 + \sigma_t \epsilon_t, \quad \sigma_t = \sigma_q \sqrt{t}, \quad \epsilon_t \sim N(0, 1)$$

$$E[x_{t-\Delta t} - x_t | x_t] = \frac{\Delta t}{t} E[x_0 - x_t | x_t] = \frac{\Delta t}{t \sigma_t} E[\epsilon_t | x_t]$$

$$\underbrace{E[X_{t-\Delta t} | x_t]}_{f(x_t, t)} = \frac{\Delta t}{t \sigma_t} \underbrace{E[\epsilon_t | x_t]}_{h(x_t, t)} + x_t$$

Summary

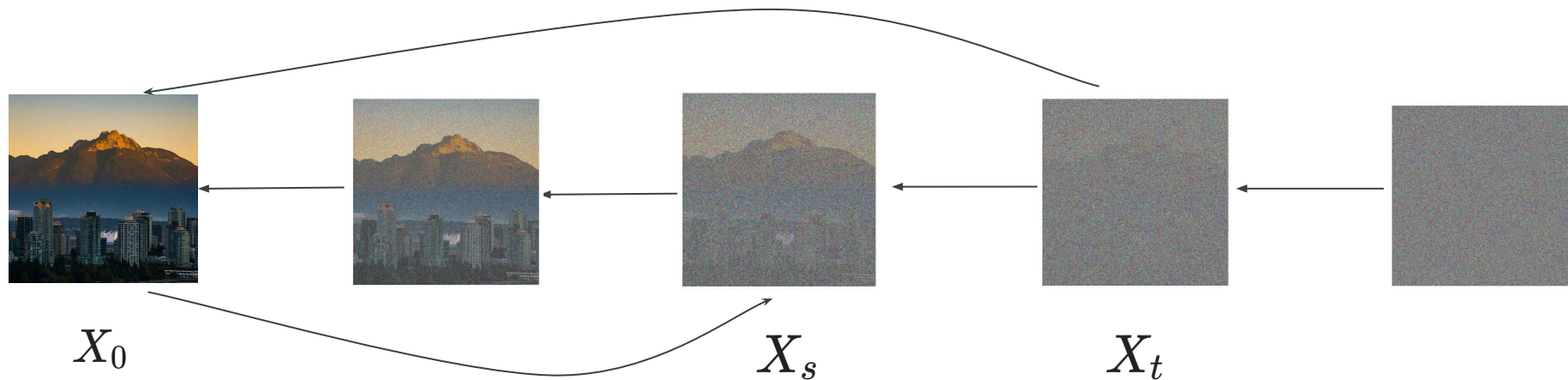
Diffusion = predict the mean, take small step, (add noise)

$$f_{\theta}(x_t, t) = E[X_{t-\Delta t} | x_t]$$

$$g_{\theta}(x_t, t) = E[X_0 | x_t]$$

$$h_{\theta}(x_t, t) = E[\epsilon_t | x_t]$$

Taking bigger steps using non-Markovian denoising process



$$p_{s|t}(x_s | x_t) = \int_{\mathbb{R}^d} \overset{\text{Prediction (will be learned)}}{p_{0|t}(x_0 | x_t)} \overset{\text{Gaussian (closed form)}}{p_{s|0,t}(x_s | x_t, x_0)} dx_0$$

Deriving the backwards distribution

$$p(x_t|x_0) \triangleq \mathcal{N}(x_t|\alpha_t x_0, \sigma_t^2)$$

Forward process

$$p(x_s|x_t, x_0) \triangleq \mathcal{N}(x_s|A_{st}x_0 + B_{st}x_t, C_{st}^2)$$

Backward process

$$p(x_s|x_0) = \int p(x_s|x_t, x_0)p(x_t|x_0)dx_t$$

Consistency

$$\Rightarrow p(x_s|x_0, x_t) = N(x_s|\mu_{s,t}(x_0, x_t), \Sigma_{s,t})$$

$$\begin{aligned}\mu_{s,t}(x_0, x_t) &= (\varepsilon^2 r_{1,2}(s, t) + (1 - \varepsilon^2)r_{0,1})x_t \\ &\quad + \alpha_s(1 - \varepsilon^2 r_{2,2}(s, t) - (1 - \varepsilon^2)r_{1,1}(s, t))x_0\end{aligned}$$

Trust me, formulae exist...

$$\Sigma_{s,t} = \sigma_s^2(1 - (\varepsilon^2 r_{1,1}(s, t) + (1 - \varepsilon^2))^2)\text{Id}$$

$$r_{i,j}(s, t) = (\alpha_t/\alpha_s)^i (\sigma_s^2/\sigma_t^2)^j$$

Conditional backwards distribution

$$p_{s|0,t}(x_s | x_0, x_t)$$

$$X_t = \alpha_t X_0 + \sigma_t X_1 + \epsilon Z_t \quad \text{Flow matching is } \epsilon=0$$

$$X_0 \sim \rho_0, X_1 \sim \rho_1, Z \sim N(0, I)$$

ρ_0 = empirical dist, ρ_1 = source dist (eg $N(0, I)$)

$$p(x_s | x_0, x_t) = N(x_s | \mu_{s,t}(x_0, x_t), \Sigma_{s,t})$$

$$\begin{aligned} \mu_{s,t}(x_0, x_t) &= (\epsilon^2 r_{1,2}(s, t) + (1 - \epsilon^2) r_{0,1}) x_t \\ &\quad + \alpha_s (1 - \epsilon^2 r_{2,2}(s, t) - (1 - \epsilon^2) r_{1,1}(s, t)) x_0 \end{aligned}$$

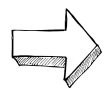
Trust me, formulae exist...

$$\Sigma_{s,t} = \sigma_s^2 (1 - (\epsilon^2 r_{1,1}(s, t) + (1 - \epsilon^2)))^2 \text{Id}$$

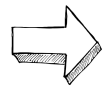
$$r_{i,j}(s, t) = (\alpha_t / \alpha_s)^i (\sigma_s^2 / \sigma_t^2)^j$$

Predicting clean distribution

$$p_{0|t}(x_0|x_t)$$

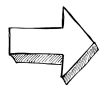


Approximate $p_{0|t}(x_0|x_t)$ with $p_{0|t}^\theta(x_0|x_t)$



Diffusion models only approximate the **conditional mean**

$$p_{0|t}^\theta(x_0|x_t) = \delta_{x_\theta(t,x_t)} \quad x_\theta(t, x_t) \approx \mathbb{E}_{x_0|x_t}[x_0 | x_t]$$



Diffusion model loss function

$$\mathcal{L}(\theta) = \int_0^1 w(t) \mathbb{E}_{x_0, x_t} [\|x_0 - x_\theta(t, x_t)\|^2] dt$$

Distributional diffusion models

*a.k.a.: going **beyond** the conditional mean approximation...*

- ➡ Standard diffusion models learn a **conditional mean**
- ➡ Approximation is **valid** only for **large NFEs** (continuous limit)
- ➡ We propose to learn the **conditional distribution**

Better approx ➡ **Less steps** ➡ **Faster sampling**

Unlike distillation, we do not need to train a separate model

A Scoring Rule Detour

How to design a “good” loss function for evaluating an implicit generative model (i.e., without using likelihoods)?

Given p and q , quantify how close those distributions are:

➡ **Scoring rule** $S(p, x)$ (quality of prediction p when we observe x)

➡ **Expected score** $S(p, q) := \mathbb{E}_q[S(p, Y)]$

Strictly proper if $S(q, q) \geq S(p, q)$ with equality iff $p = q$

➡ **Kernel score** $S_\rho(p, y) = \underbrace{\frac{1}{2} \mathbb{E}_{p \otimes p}[\rho(X, X')]}_{\text{Interaction term}} - \underbrace{\mathbb{E}_p[\rho(X, y)]}_{\text{Confinement term}}$

➡ **Generalized Kernel score** $S_{\lambda, \rho}(p, y) = \underbrace{\frac{\lambda}{2} \mathbb{E}_{p \otimes p}[\rho(X, X')]}_{\text{Balance interaction/confinement}} - \mathbb{E}_p[\rho(X, y)]$

A Scoring Rule Detour

How to design a “good” loss function for evaluating an implicit generative model (i.e., without using likelihoods)?

Given p and q , quantify how close those distributions are:

➡ **Scoring rule** $S(p, x)$ (quality of prediction p when we observe x)

➡ **Generalized kernel score**

$$S(p, x) = \underbrace{E_p[k(X, x)]}_{\text{confinement}} - \frac{\lambda}{2} \underbrace{E_{p \times p}[k(X, X')]}_{\text{interaction}}$$

$$k_{\text{rbf}}(x, x') = \exp \left[-\|x - x'\|^2 / 2\sigma^2 \right]$$

$$k_{\text{eng}}(x, x') = -\|x - x'\|^\beta$$

Generalized Kernel Score

$$S_{\lambda, \rho}(p, y) = \frac{\lambda}{2} \mathbb{E}_{p \otimes p}[\rho(X, X')] - \mathbb{E}_p[\rho(X, y)]$$

How to pick your **score**? Define a **kernel** and set $\rho = -k$

$$k_{\text{imq}}(x, x') = (\|x - x'\|^2 + c)^{-1/2}$$

$$k_{\text{rbf}}(x, x') = \exp[-\|x - x'\|^2 / 2\sigma^2]$$

$$k_{\text{exp}}(x, x') = \exp[-\|x - x'\| / \sigma]$$

$$k_{\text{eng}}(x, x') = -\|x - x'\|^\beta$$

*All those kernels imply a **strictly proper scoring rule***

energy distance kernel

(will be easy to relate to diffusion models)

Distributional Diffusion Loss

$$\left[\mathcal{L}(\theta) = \int_0^1 w(t) \mathbb{E}_{x_0, x_t} [\|x_0 - x_\theta(t, x_t)\|^2] dt \right] \quad \left[\mathcal{L}(\theta) = - \int_0^1 w(t) \mathbb{E}_{x_0, x_t} \left[S_{\lambda, \beta}(p_{0|t}^\theta(\cdot|x_t), x_0) \right] dt, \right]$$

Diffusion model loss

Distributional Diffusion model loss

$$S_{\lambda, \beta}(p_{0|t}^\theta(\cdot|x_t), x_0) = \frac{\lambda}{2} \mathbb{E}_{p_{0|t}^\theta(\cdot|x_t) \otimes p_{0|t}^\theta(\cdot|x_t)} [\|X - X'\|^\beta] - \mathbb{E}_{p_{0|t}^\theta(\cdot|x_t)} [\|X - x_0\|^\beta]$$

*Generated samples should **not be** too close*

*Generated samples **should be** close to the data*

some special cases...

$\beta = 2, \lambda = 0$ – Recover **diffusion model loss**

$\beta \in (0, 2), \lambda = 1$ – Recover **strictly proper scoring rules**

Training

$$p(x_t|x_0) = \mathcal{N}(x_t; \alpha_t x_0, \sigma_t^2 \text{Id}), \quad (2)$$

$$\mathcal{L}(\theta) = - \int_0^1 w(t) \mathbb{E}_{x_0, x_t} \left[S_{\lambda, \beta}(p_{0|t}^\theta(\cdot|x_t), x_0) \right] dt,$$

$$S_{\lambda, \beta}(p_{0|t}^\theta(\cdot|x_t), x_0) = \frac{\lambda}{2} \mathbb{E}_{p_{0|t}^\theta(\cdot|x_t) \otimes p_{0|t}^\theta(\cdot|x_t)} [\|X - X'\|^\beta] - \mathbb{E}_{p_{0|t}^\theta(\cdot|x_t)} [\|X - x_0\|^\beta]$$

$$\mathcal{L}_{n,m}(\theta) = \frac{1}{nm} \sum_{i,j=1}^{n,m} w_{t_i} \left[\left\| X_0^i - \hat{x}_\theta(t_i, X_{t_i}, \xi_i^j) \right\|^\beta \right. \quad (14)$$

$$\left. \frac{-\lambda}{2(m-1)} \sum_{j' \neq j}^m \left\| \hat{x}_\theta(t_i, X_{t_i}, \xi_i^j) - \hat{x}_\theta(t_i, X_{t_i}, \xi_i^{j'}) \right\|^\beta \right].$$

Training

Algorithm 1 Distributional Diffusion Model (training)

Require: M training steps, schedule (α_t, σ_t) , distribution

p_0 , weights θ_0 , batch size n , population size m

for $k = 1 : M$ **do**

 Sample $t_i \sim \text{Unif}([0, 1])$ for $i \in [n]$

 Sample $X_0^i \stackrel{\text{i.i.d.}}{\sim} p_0$ for $i \in [n]$

 Sample $X_{t_i}^i \sim p_{t_i|0}(\cdot|X_0^i)$ for $i \in [n]$ using (2)

 Sample $\xi_{i,j} \sim \mathcal{N}(0, \text{Id})$ for $i \in [n], j \in [m]$

 Set $\theta_k = \theta_{k-1} - \delta \nabla \mathcal{L}_{n,m}(\theta)|_{\theta_{k-1}}$ using (14)

end for

Return: θ_M

$$p(x_t|x_0) = \mathcal{N}(x_t; \alpha_t x_0, \sigma_t^2 \text{Id}), \quad (2)$$

$$\mathcal{L}_{n,m}(\theta) = \frac{1}{nm} \sum_{i,j=1}^{n,m} w_{t_i} \left[\left\| X_0^i - \hat{x}_\theta(t_i, X_{t_i}, \xi_i^j) \right\|^\beta \right. \\ \left. - \frac{\lambda}{2(m-1)} \sum_{j' \neq j}^m \left\| \hat{x}_\theta(t_i, X_{t_i}, \xi_i^j) - \hat{x}_\theta(t_i, X_{t_i}, \xi_i^{j'}) \right\|^\beta \right]. \quad (14)$$

Architecture for generating clean sample

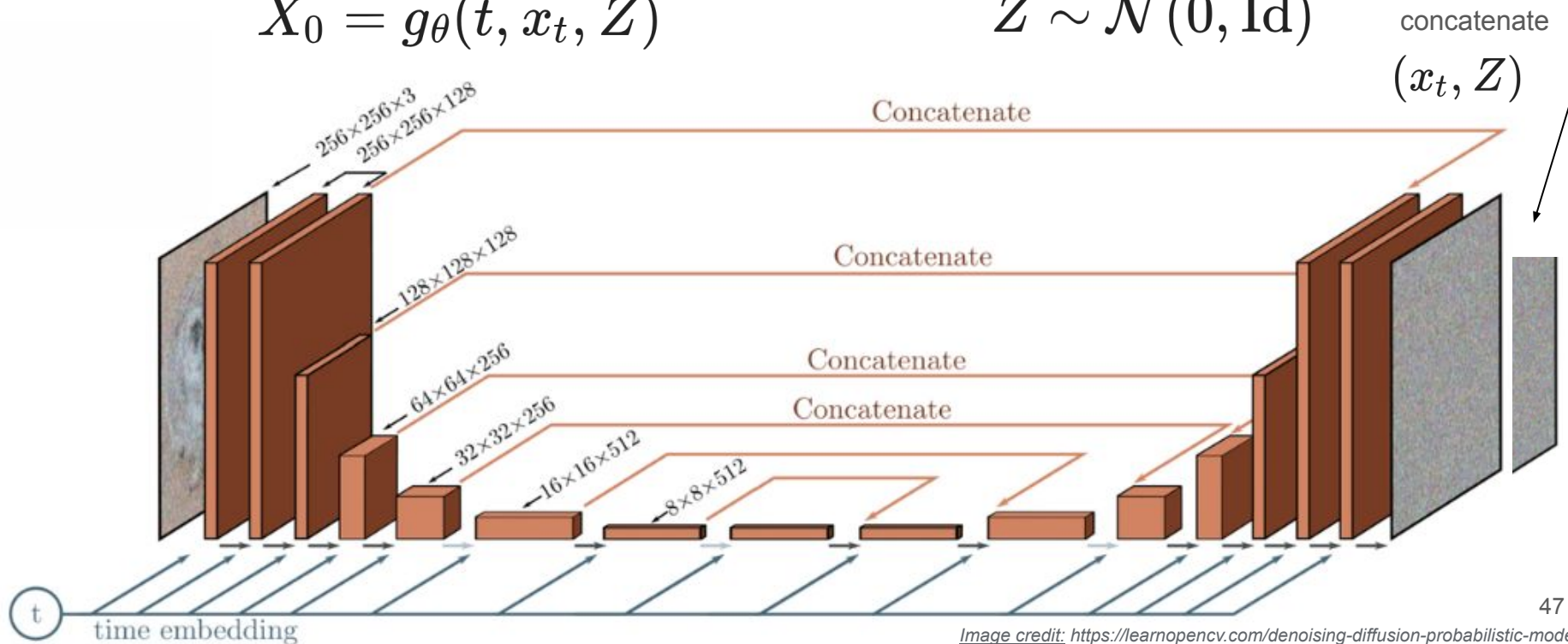
How do we parameterize $X_0 \sim p_{0|t}^\theta(x_0|x_t)$?

→ Res. block
→ Res. block + self-attention

Classical **U-Net generator** with **additional noise**

$$X_0 = g_\theta(t, x_t, Z)$$

$$Z \sim \mathcal{N}(0, \text{Id})$$



Architecture for generating clean sample

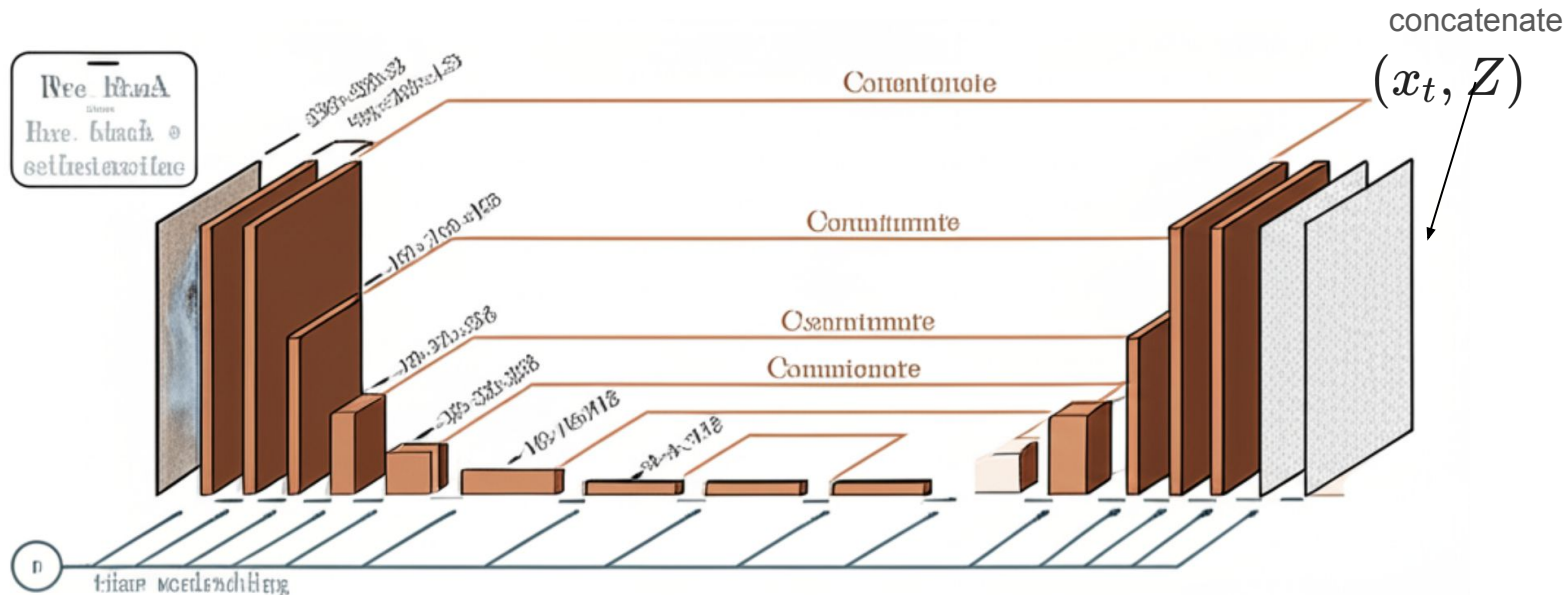
How do we parameterize $X_0 \sim p_{0|t}^\theta(x_0|x_t)$?

Res. block
Res. block +
self-attention

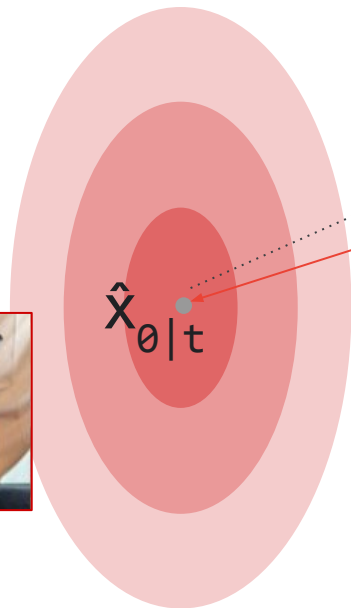
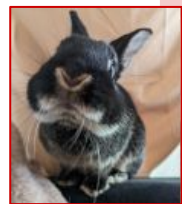
Classical **U-Net generator** with **additional noise**

$$X_0 = g_\theta(t, x_t, Z)$$

$$Z \sim \mathcal{N}(0, \text{Id})$$



Inference (generation)

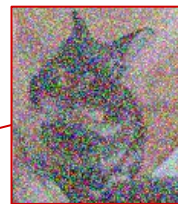


$$p(x_s | x_t) = \int_{\mathbb{R}^d} p(x_s | x_0, x_t) p(x_0 | x_t) dx_0.$$

x_s

•

ξ



x_t

Z

g

Algorithm 2 Distributional Diffusion Model (sampling)

Require: $\{t_k\}_{k=0}^N$ with $t_0 = 0 < \dots < t_N = 1$, churn parameter ε

Sample $X_{t_N} \sim \mathcal{N}(0, \text{Id})$

for $k \in \{N-1, \dots, 0\}$ **do**

 Sample $\xi \sim \mathcal{N}(0, \text{Id})$

 Sample $Z \sim \mathcal{N}(0, \text{Id})$

 Set $\hat{X}_0 = \hat{x}_\theta(t_{k+1}, X_{t_{k+1}}, \xi)$

 Compute $\mu_{t_k, t_{k+1}}(\hat{X}_0, X_{t_{k+1}}), \Sigma_{t_k, t_{k+1}}^{1/2}$ using (4)

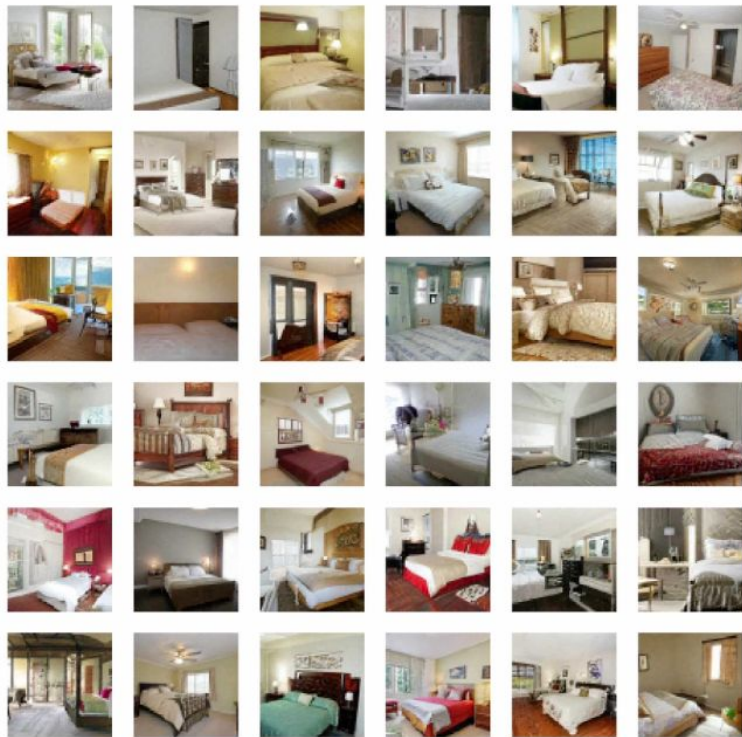
 Set $X_{t_k} = \mu_{t_k, t_{k+1}}(\hat{X}_0, X_{t_{k+1}}) + \Sigma_{t_k, t_{k+1}}^{1/2} Z$

end for

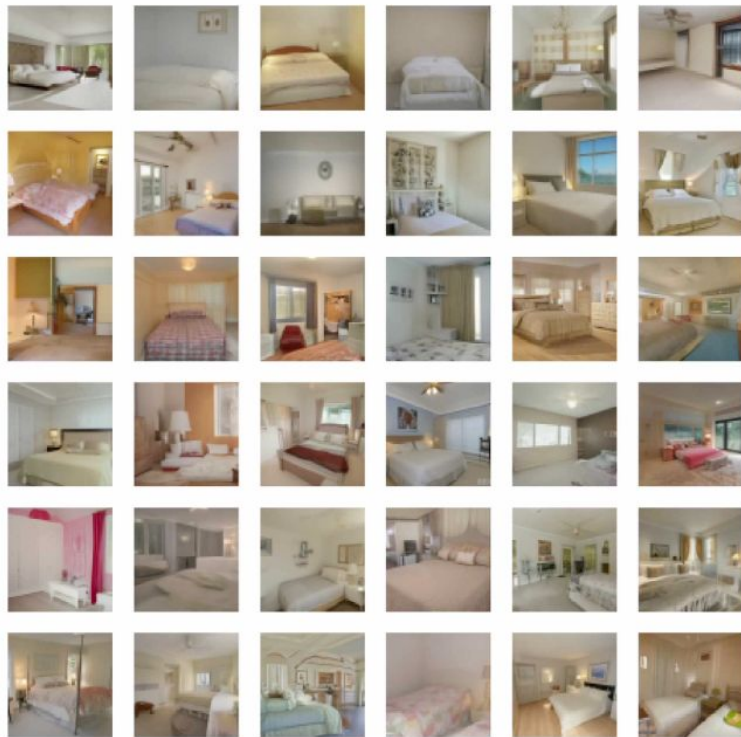
Return: X_0

Results on LSUN Bedrooms (10 steps)

Distributional

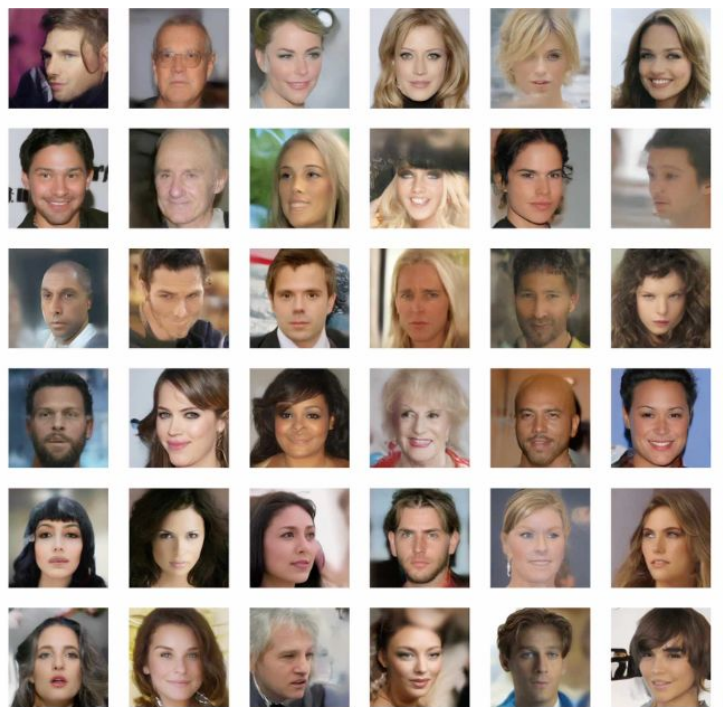


Standard

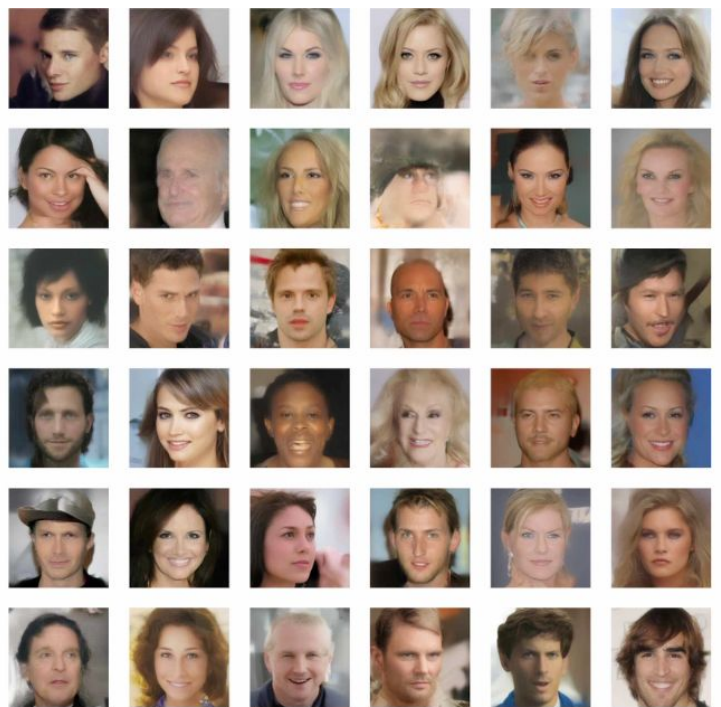


Results on latent CelebA HQ (10 steps)

Distributional

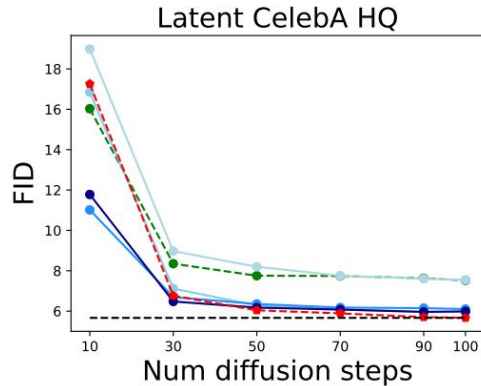
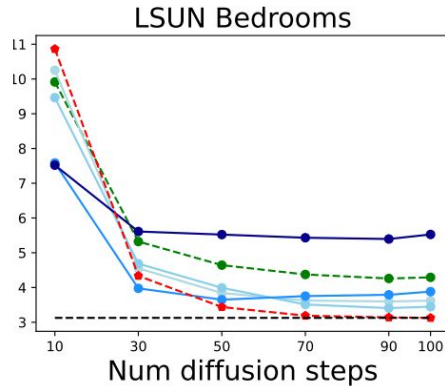
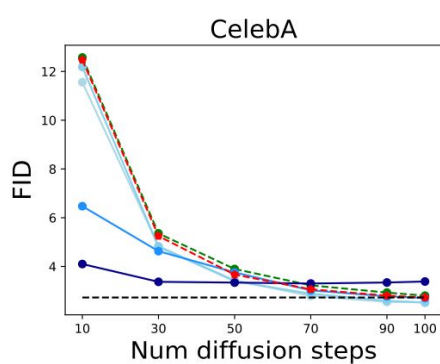
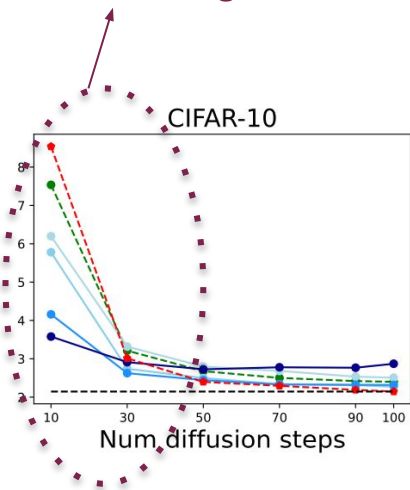


Standard



FID vs num. steps

Low NFE regime



Legend: $\beta = 2, \lambda = 0$ (green dashed line with dots), $\beta < 2, \lambda = 0$ (light blue dashed line with dots), $\beta < 2, \lambda = 0.1$ (medium blue dashed line with dots), $\beta < 2, \lambda = 0.5$ (dark blue dashed line with dots), $\beta < 2, \lambda = 1$ (dark blue solid line with dots), Diffusion (red dashed line with dots).

TL;DR distributional diffusion models **outperform** conventional diffusion models in the **low NFE regime**.

They coincide with conventional diffusion models in the **high NFE regime**.

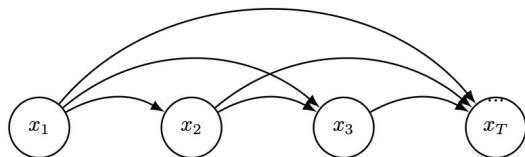
Speeding up autoregressive models

ICML 2023

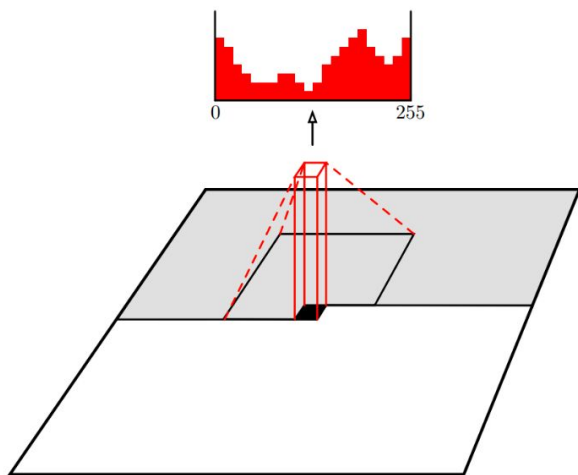
Muse: Text-To-Image Generation via Masked Generative Transformers

Huiwen Chang, Han Zhang, Jarred Barber, AJ Maschinot, Jose Lezama, Lu Jiang, Ming-Hsuan Yang, Kevin Murphy, William Freeman, Michael Rubinstein, Yuanzhen Li, Dilip Krishnan

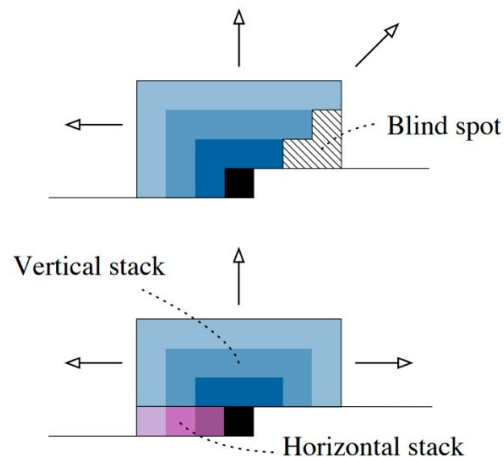
Autoregression in pixel space: PixelRNN and PixelCNN



1d AR model



1	1	1	1	1
1	1	1	1	1
1	1	0	0	0
0	0	0	0	0
0	0	0	0	0

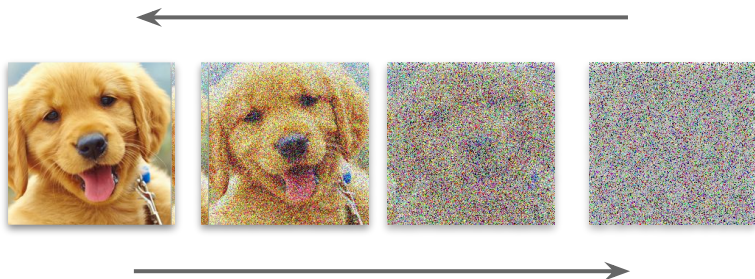


van den Oord et al. 'Pixel Recurrent Neural Networks' (2016)
van den Oord et al. 'Conditional Image Generation with PixelCNN Decoders' (2016)

Continuous vs Discrete Diffusion

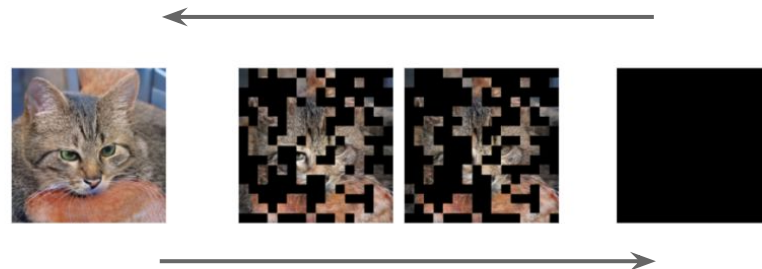
Continuous Diffusion

(e.g. Imagen-3, Dall-E 3)



Forward process is a destructive process by adding Gaussian noise

Discrete Diffusion

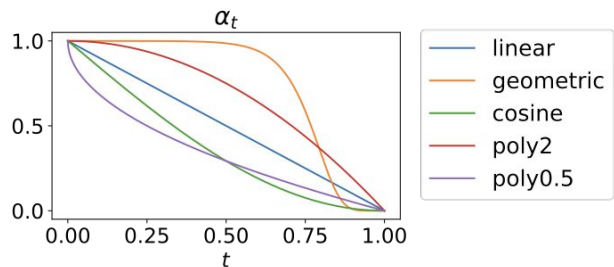
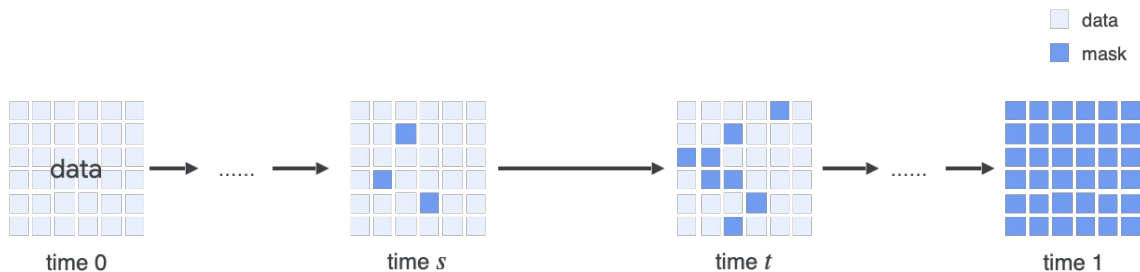


Forward pass is the absorbing state diffusion process in **image token space**

[Discrete Predictor-Corrector Diffusion Models for Image Synthesis](#), Lezama et. al. 2023
[Simplified and Generalized Masked Diffusion for Discrete Data](#), Shi et al, 2024

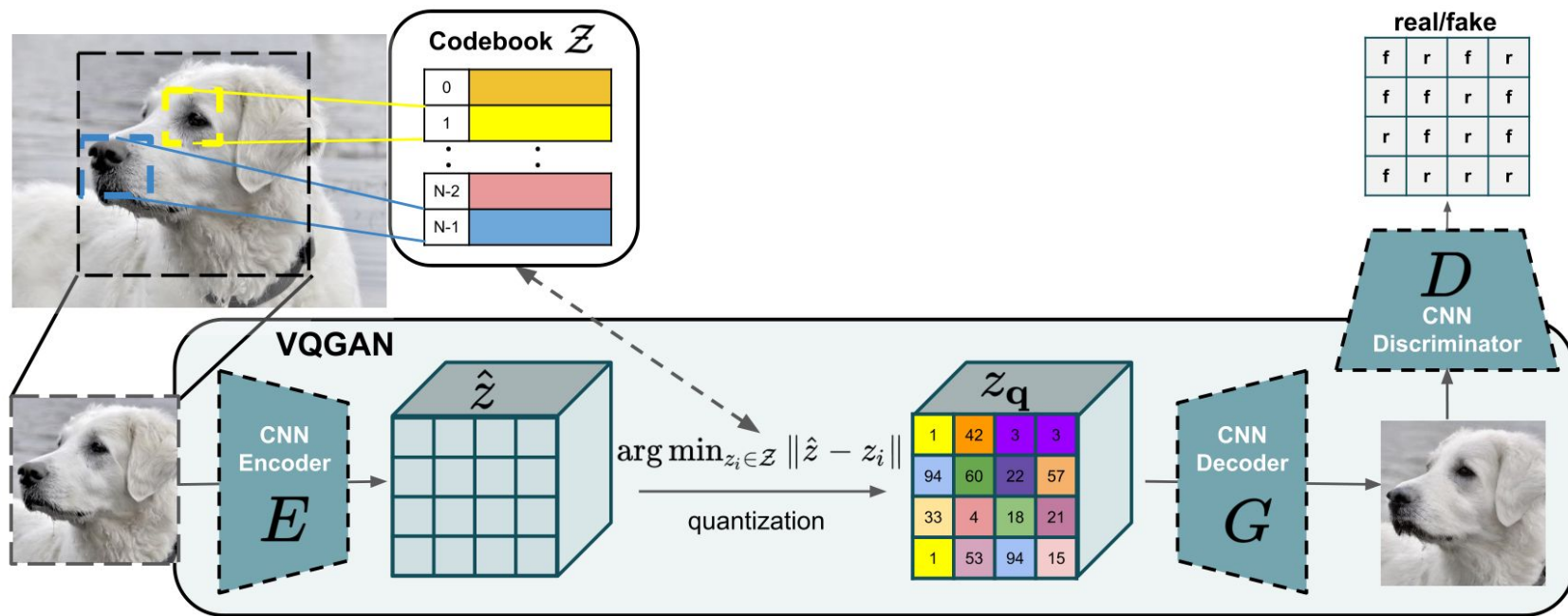
Masked discrete diffusion

- **Masked diffusion**: Also known as absorbing diffusion, first proposed in Austin et al. (2021)
- Generative modeling as iterative unmasking



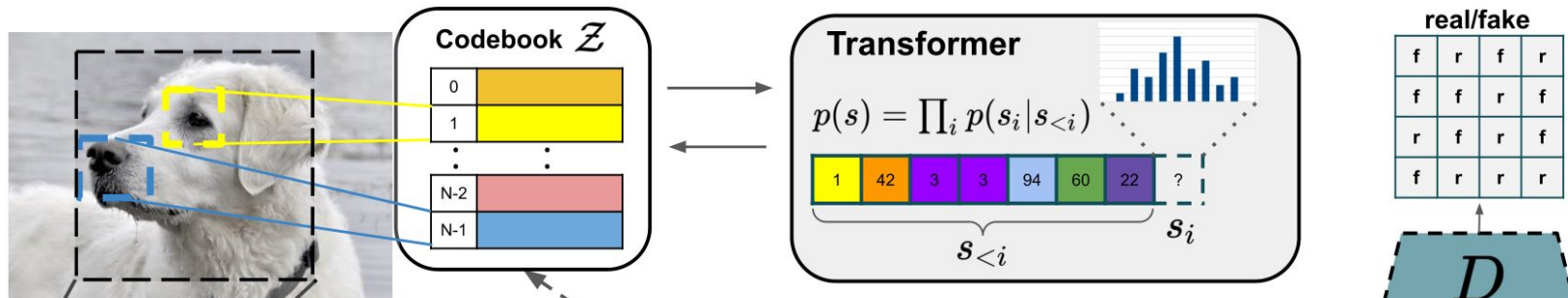
Masking schedule α_t : The expected proportion of unmasked tokens at t

Learning discrete latent space: VQ-VAE/VQ-GAN



van den Oord et al. 'Neural Discrete Representation Learning' (2017)
Esser et al. 'Taming Transformers for High-Resolution Image Synthesis' (2020)

Autoregression in latent space: VQGAN + AR transformer



Autoregressive transformer model

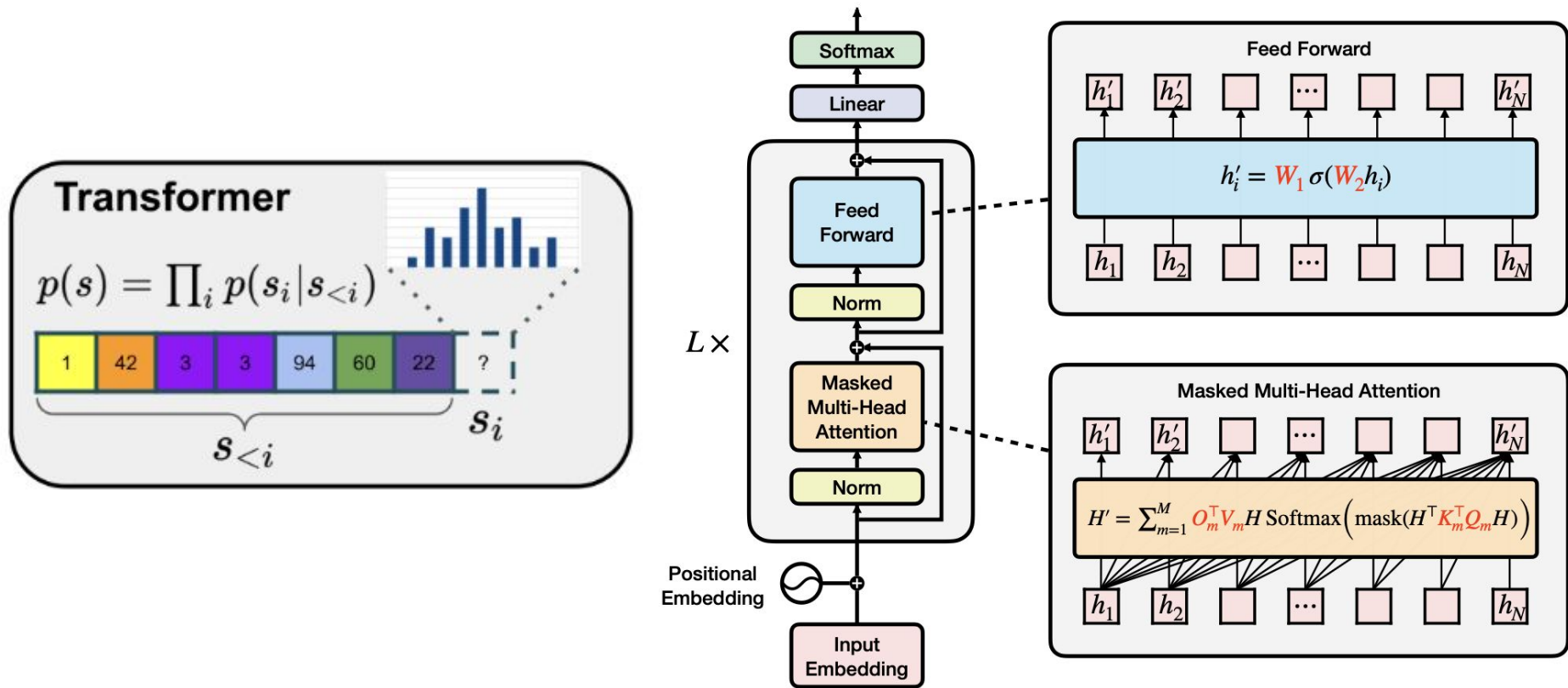
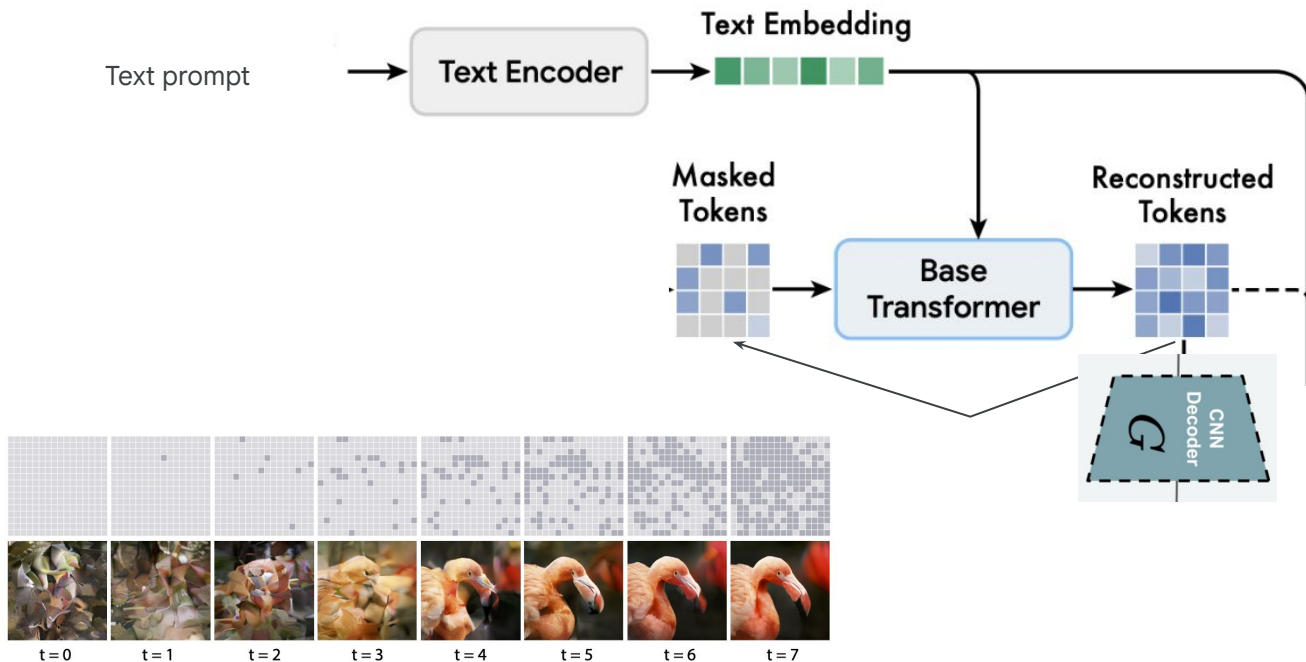


Figure from "An Overview of Large Language Models for Statisticians", Ji et al., 2025

Muse: <https://muse-model.github.io/>

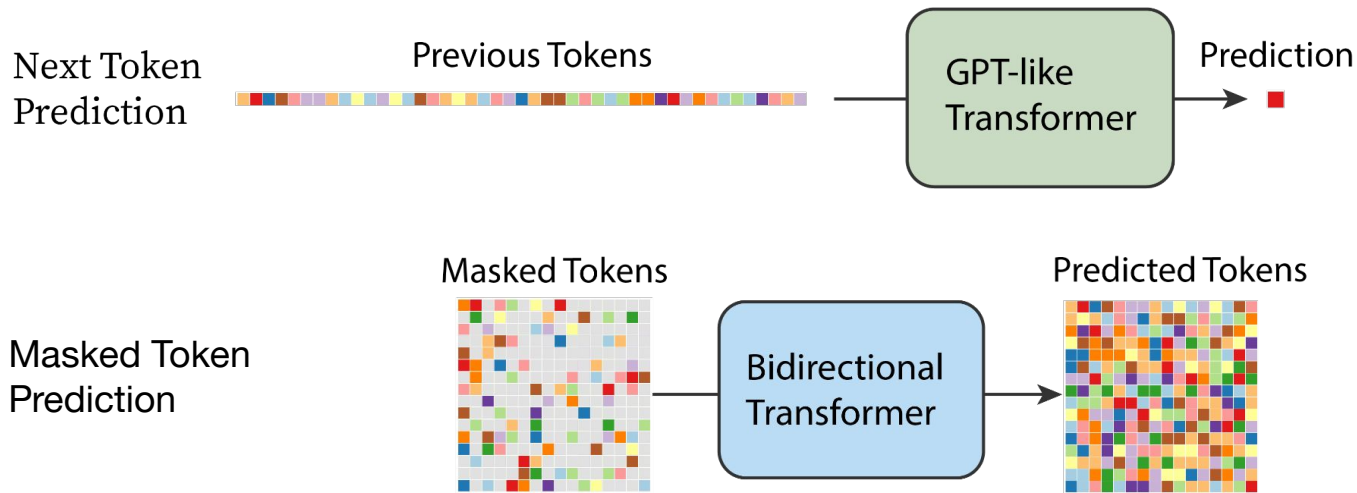
Proprietary + Confidential

- Builds on MaskGIT (Masked Generative Image Transformer)
- Key idea: Generate multiple discrete latent tokens in parallel



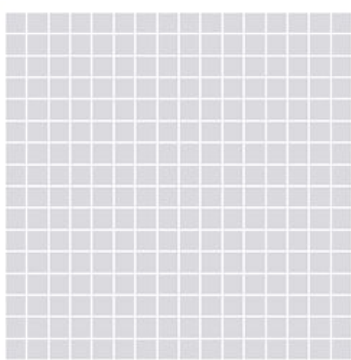
Three differences from autoregressive models

1. Bi-directional attention mechanism; all tokens attend to all others

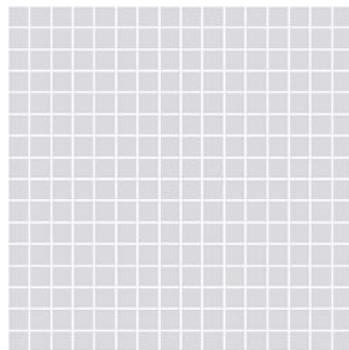


Three differences from autoregressive models

1. Bi-directional attention mechanism; all tokens attend to all others
2. Parallel decoding enables much faster sampling



Autoregressive Models (e.g. Parti, DALL·E)
decodes in a raster scan order



Our parallel decoding is >20x faster

Three differences from autoregressive models

1. Bi-directional attention mechanism; all tokens attend to all others.
2. Parallel decoding enables much faster sampling speed.
3. Random masking during training enables zero-shot editing ability.



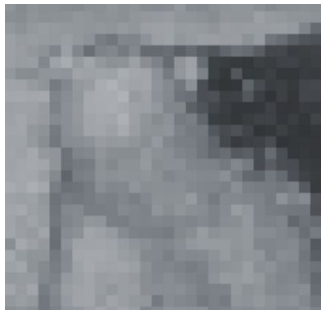
Inpainting



Extrapolation



in any directions

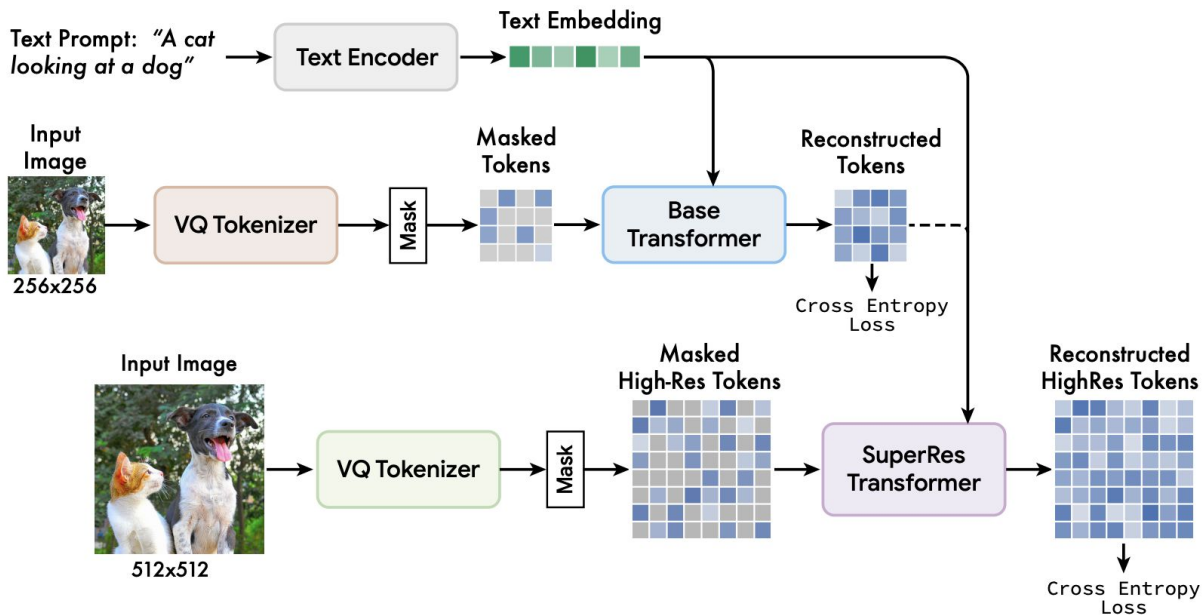


Editing based on
attention

Full Muse Model

Four components:

1. Pre-trained Text Encoder
2. Image Tokenizer
3. Base text-to-image generative transformer
4. SuperRes text-to-image generative transformer



Muse Model: 1/4

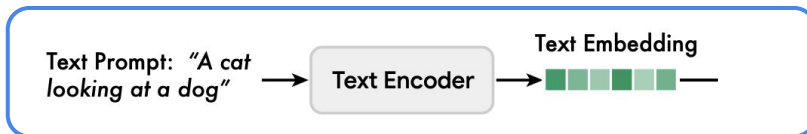
Four components:

1. **Pre-trained Text Encoder**

2. Image Tokenizer

3. Base text-to-image generative transformer

4. SuperRes text-to-image generative transformer

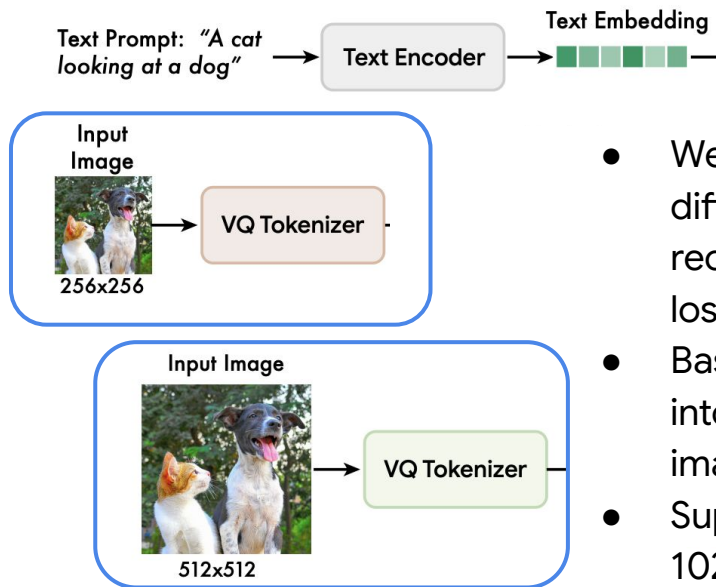


- T5-XXL pre-trained text model (4.6B parameters)
- Encodes text prompt into a sequence of 4096-D text embedding
- Chosen since it was used by the prior Imagen diffusion model

Muse Model: 2/4

Four components:

1. Pre-trained Text Encoder
2. **Image Tokenizer**
3. Base text-to-image generative transformer
4. SuperRes text-to-image generative transformer

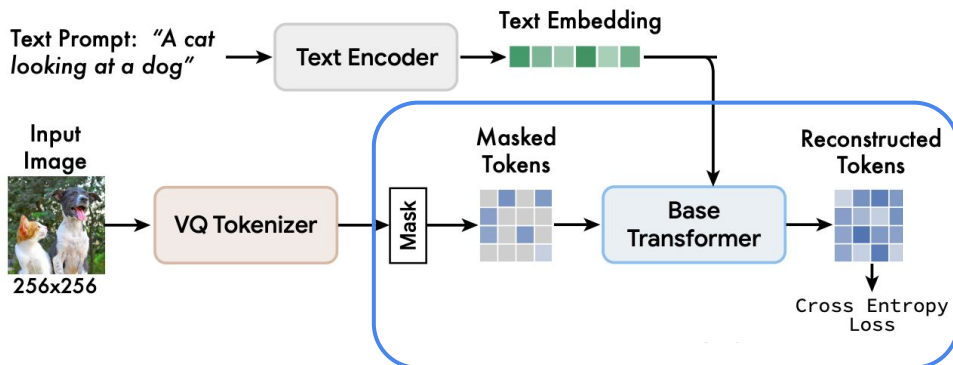


- We pre-train two VQ tokenizers at different resolution with image reconstruction loss, quantization loss, and GAN loss
- Base: Compress 256x256 images into 16x16 perceptual discrete image tokens
- Super-res: Compress 512x512 or 1024x1024 images into 64x64 perceptual discrete image tokens

Muse Model: 3/4

Four components:

1. Pre-trained Text Encoder
2. Image Tokenizer
3. **Base text-to-image generative transformer**
4. SuperRes text-to-image generative transformer

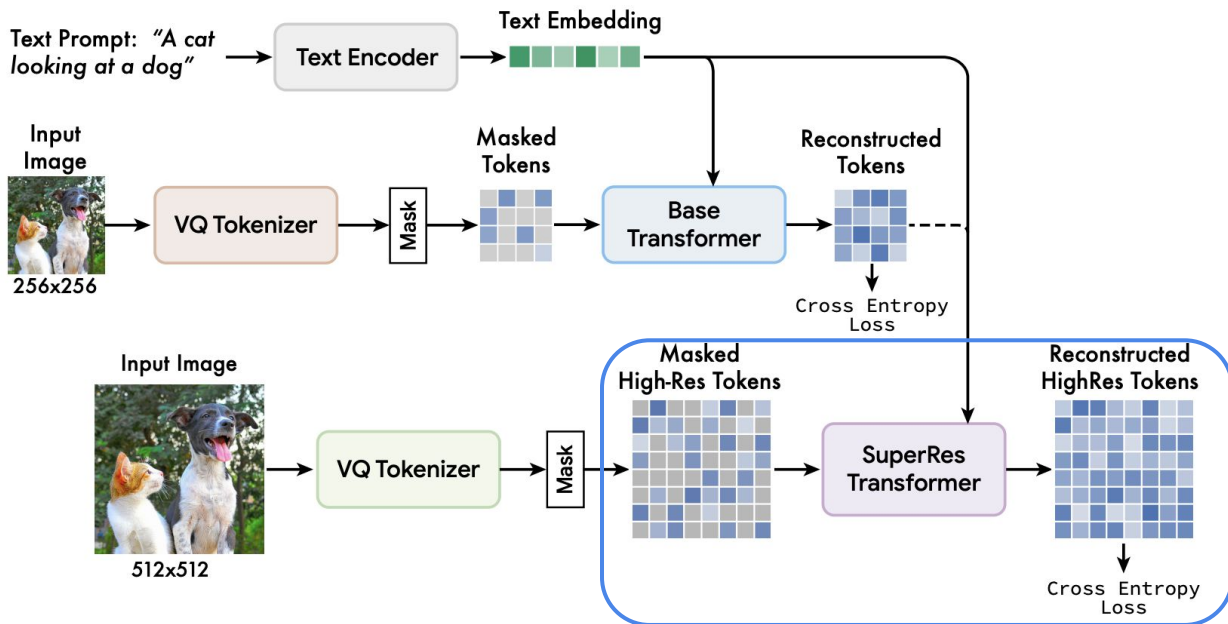


- We used "Masked Language Modeling" objective (cf BERT).
- During training, we first randomly sample a masking ratio in $(0,1]$, to select masked tokens from the sequence
- Text embedding is projected and then attended to image tokens in the cross attention
- We use cross entropy loss on masked tokens

Muse Model: 4/4

Four components:

1. Pre-trained Text Encoder
2. Image Tokenizer
3. Base text-to-image generative transformer
4. **SuperRes text-to-image generative transformer**



We find that directly predicting 64x64 tokens leads to worse text alignment, and cascaded models seem to mitigate this issue

Comparisons to other SOTA models (c. 2023)

A high contrast portrait of a very happy fuzzy panda dressed as a chef in a high end kitchen making dough. There is a painting of flowers on the wall behind him.

DALL-E 2



Imagen



MUSE

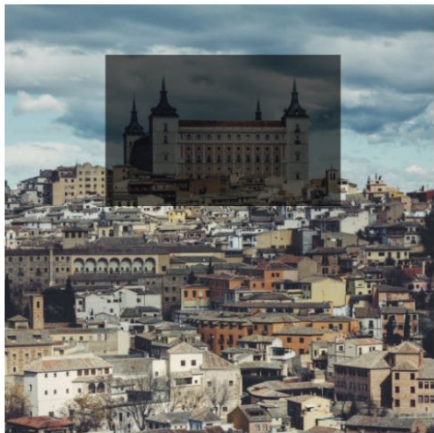


A stack of 3 books. A green book is on the top, sitting on a red book. The red book is in the middle, sitting on a blue book. The blue book is on the bottom.

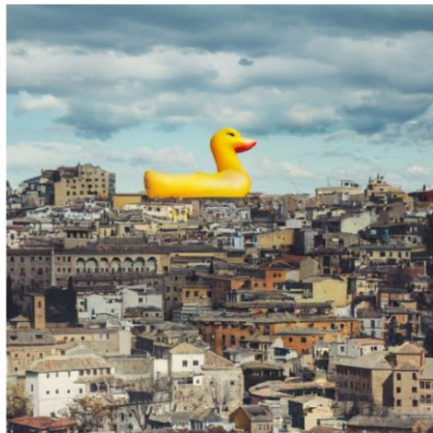


Text Guided Inpainting (no fine-tuning)

Input



Output



A funny big inflatable yellow duck



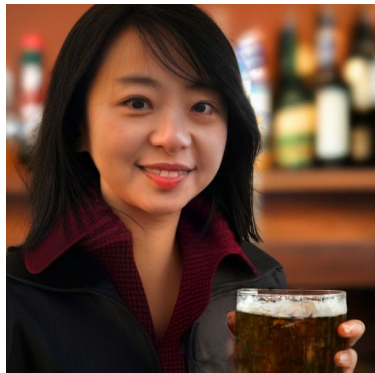
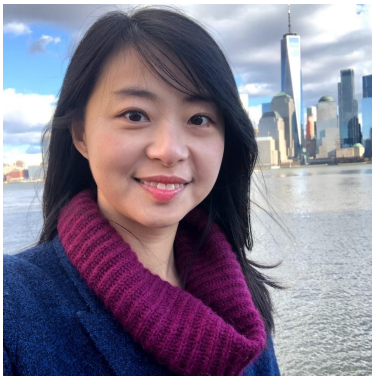
Hot air balloons



A futuristic Streamline Moderne building

Inpainting

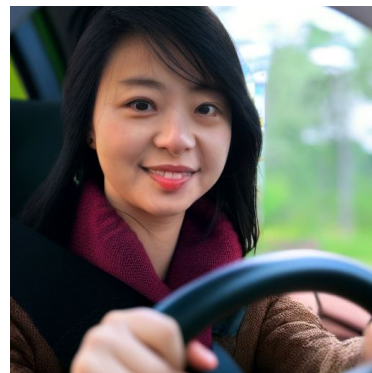
Text Guided Outpainting (Uncropping)



[Huiwen] in a bar



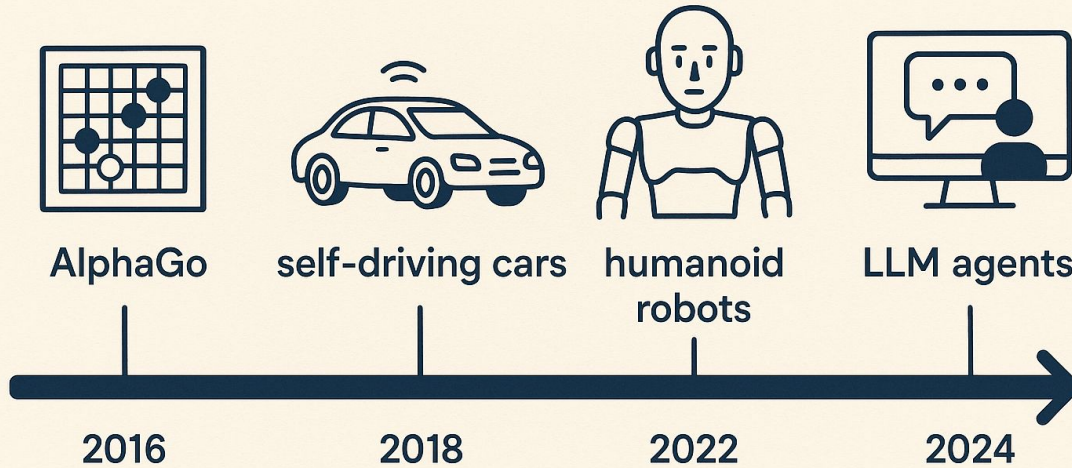
[Huiwen] as a sushi
chef



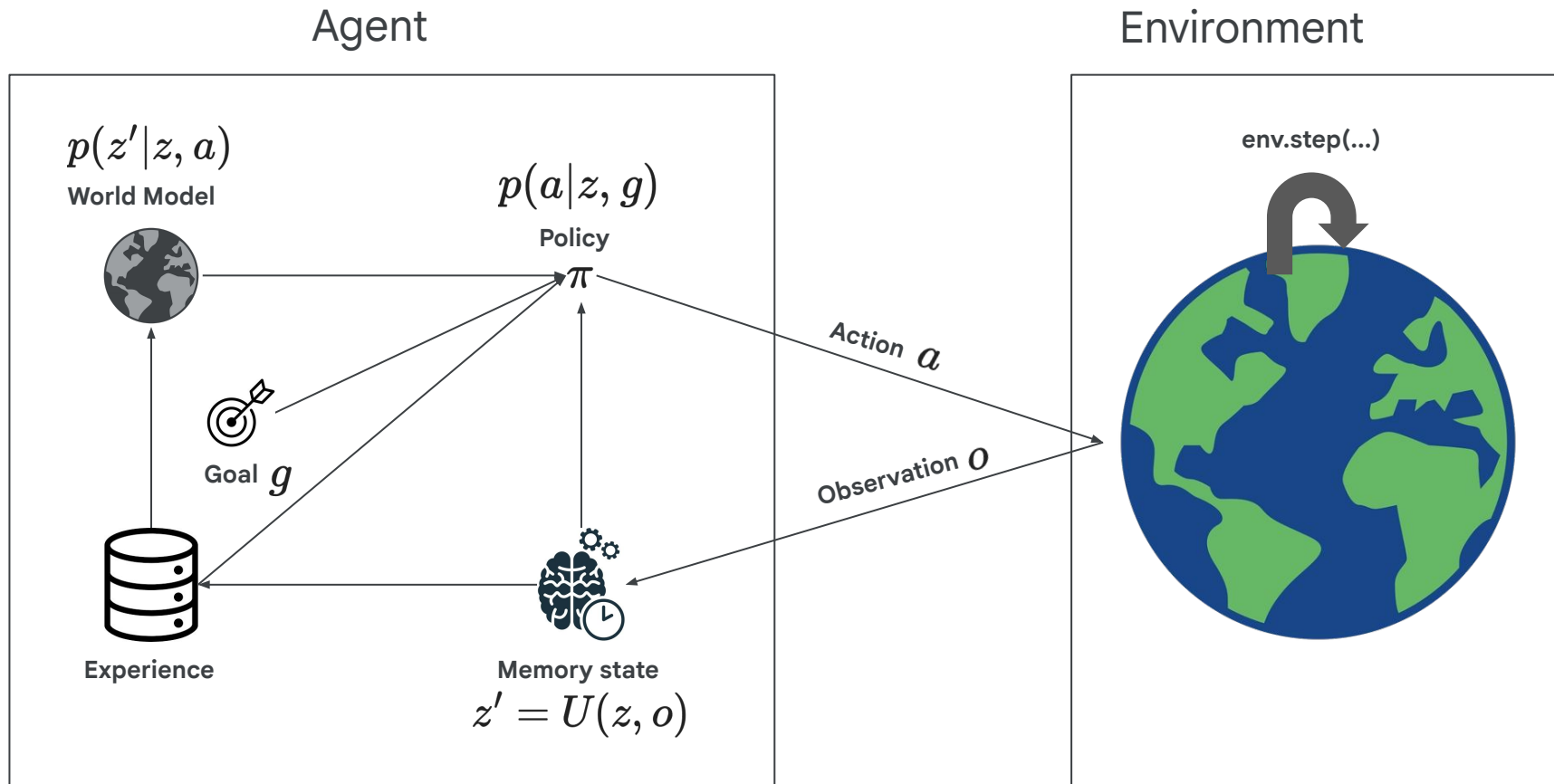
[Huiwen] driving

The Agentic AI revolution

The Rise of Agentic AI



Agentic AI: a probabilistic perspective



This talk: Generative models for policies and world models

Faster diffusion policies (ICML 2025)

Distributional Diffusion Models with Scoring Rules

Valentin De Bortoli, Alexandre Galashov, J. Swaroop Guntupalli, Guangyao Zhou, Kevin Murphy, Arthur Gretton, Arnaud Doucet

Multi-step diffusion world model for decision-time planning (TMLR 2025)

Diffusion Model Predictive Control

Guangyao Zhou, Sivaramakrishnan Swaminathan, Rajkumar Vasudeva Raju, J. Swaroop Guntupalli, Wolfgang Lehrach, Joseph Ortiz, Antoine Dedieu, Miguel Lázaro-Gredilla, Kevin Murphy

Latent world model for background planning (ICML 2025)

[Submitted on 3 Feb 2025 (v1), last revised 16 Jul 2025 (this version, v3)]

Improving Transformer World Models for Data-Efficient RL

Antoine Dedieu, Joseph Ortiz, Xinghua Lou, Carter Wendelken, Wolfgang Lehrach, J Swaroop Guntupalli, Miguel Lázaro-Gredilla, Kevin Patrick Murphy

Faster diffusion policies (ICML 2025)

Distributional Diffusion Models with Scoring Rules

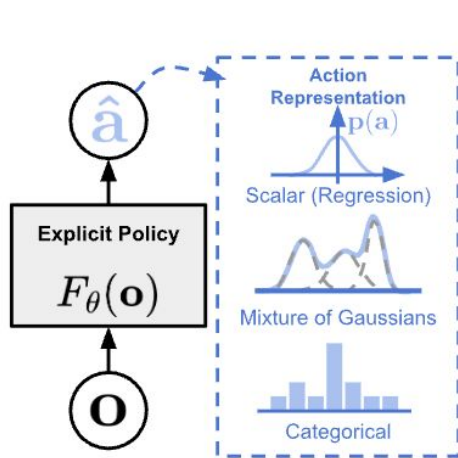
Valentin De Bortoli, Alexandre Galashov, J. Swaroop Guntupalli, Guangyao Zhou, Kevin Murphy, Arthur Gretton, Arnaud Doucet

Diffusion policies (for robotics)

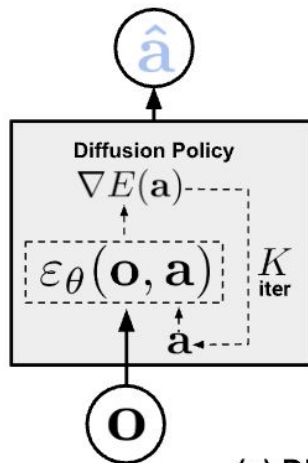
A policy is a generative model over (a sequence of) actions given state

$$\pi(a_{t:t+H} | z_t)$$

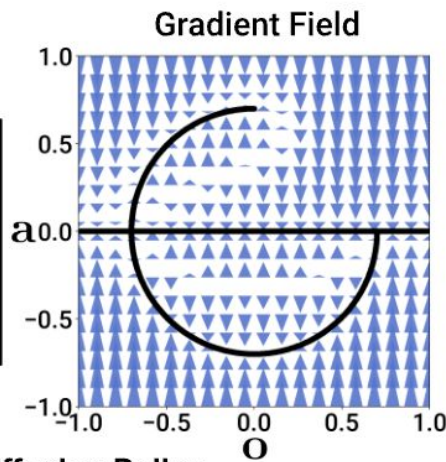
For high-dimensional continuous actions and/or long sequences, this is like a short but wide image generation problem. So diffusion is a natural choice.



(a) Explicit Policy



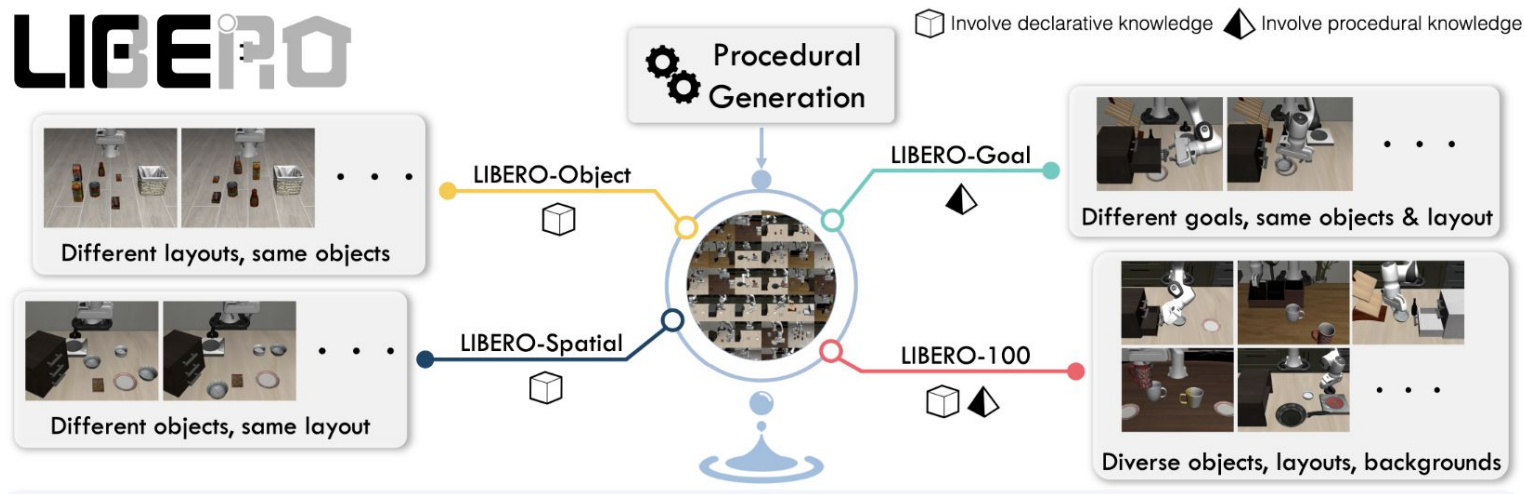
(c) Diffusion Policy



Distributional diffusion policies (for robotics)

Train $p(a_t^i | a_t^i, z)$ using behavior cloning plus distributional loss.

Action space is 7-dim, chunk horizon is 8. Input z is vision, proprio, text.
Image encoder: ResNet. Text encoder: BERT. Denoiser: transformer.



Libero goal



Libero object

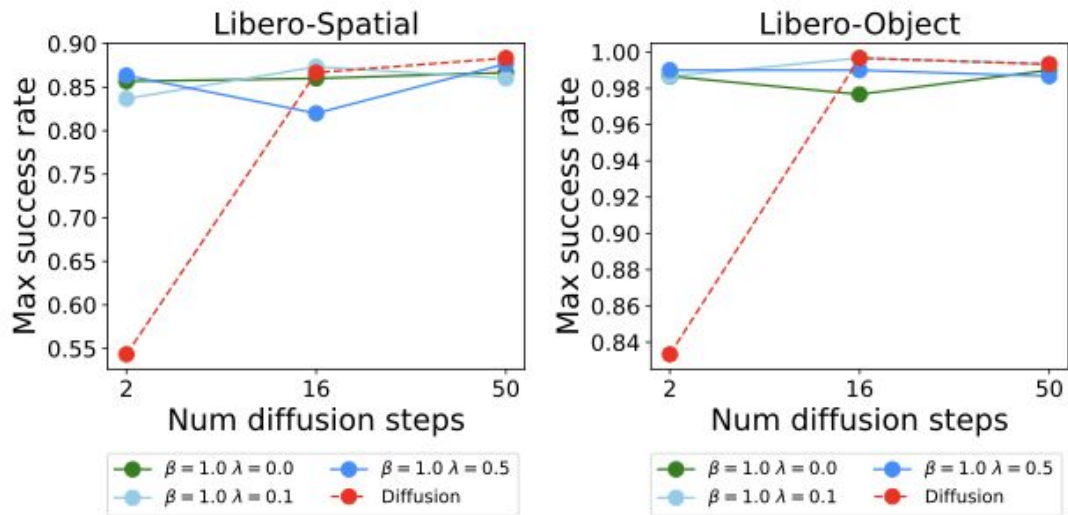


Libero spatial



Distributional diffusion policies on Libero benchmark

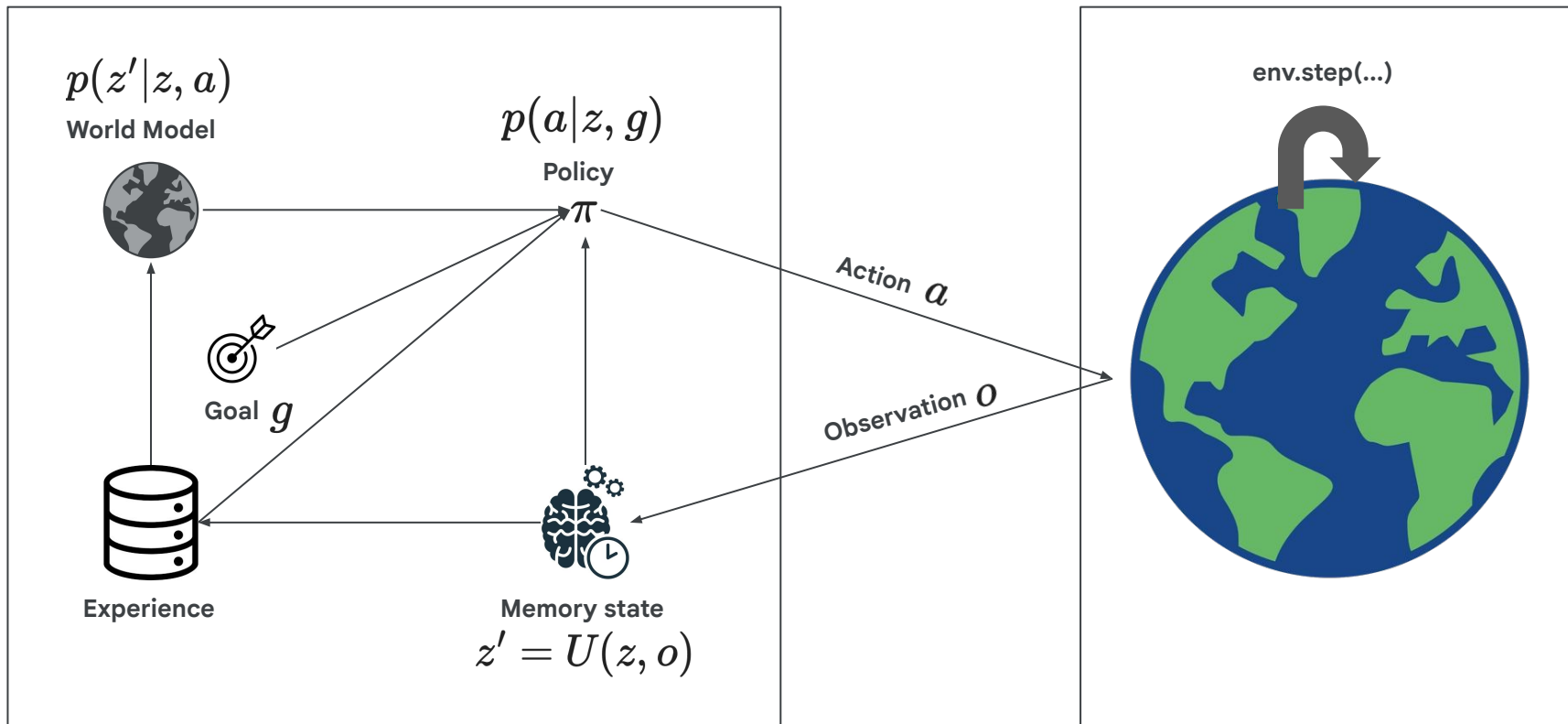
Distributional policy needs fewer steps than standard diffusion.



From behavior cloning to reinforcement learning

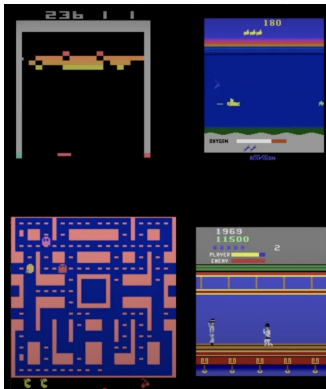
Agent

Environment



RL is data hungry and (mostly) only works in simulated environments

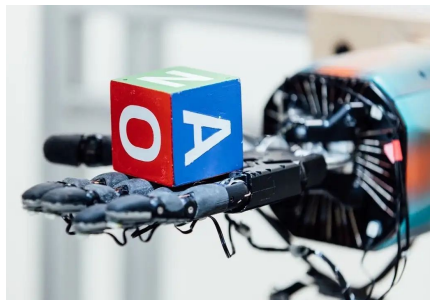
DQN (2015) from DeepMind beats humans at multiple Atari Games



AlphaGo (2016) from DeepMind beats human grandmaster at Go



Dactyl (2018) from OpenAI learns dexterous hand control

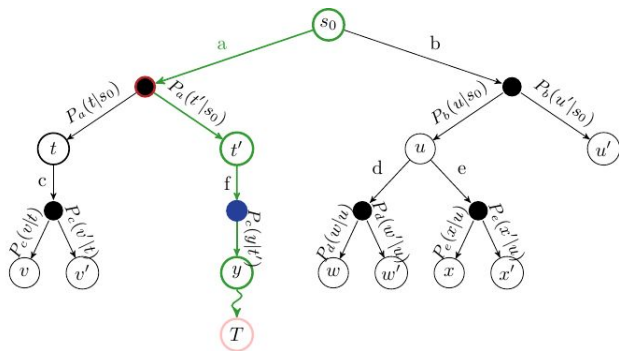


- Model-based RL to the rescue:
 - learn a simulator (a world model)
 - Then use WM to choose actions

Ways to use World Model

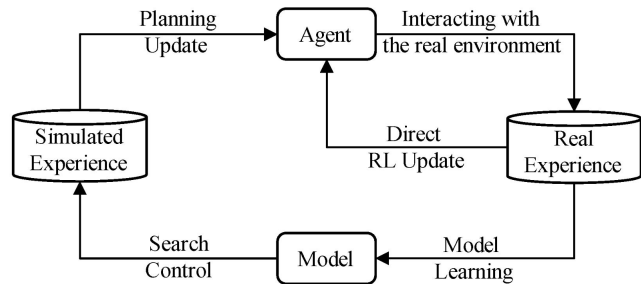
Decision time planning

- + Can do an adaptive amount of planning.
- + Can be used with any new reward fn, e.g. intrinsic reward from LLM.
- + Less sensitive to model error; results in exploration.
- Requires slow planning at test time.



Background planning

- + Planning cost is amortized into a policy.
- + Can improve data efficiency wrt environment interactions.
- A bad model can pollute the policy if the generated trajectories are OOD.
- ? Can be combined with learning from real data.



Relevant paper:

Alver, Sifa, and Doina Precup. 2022. "A Look at Value-Based Decision-Time vs. Background Planning Methods across Different Settings." *arXiv [Cs.LG]*. *arXiv*. <http://arxiv.org/abs/2206.08442>.

Multi-step diffusion world model for decision-time planning (TMLR 2025)

Diffusion Model Predictive Control

Guangyao Zhou, Sivaramakrishnan Swaminathan, Rajkumar Vasudeva Raju, J. Swaroop Guntupalli, Wolfgang Lehrach, Joseph Ortiz, Antoine Dedieu, Miguel Lázaro-Gredilla, Kevin Murphy

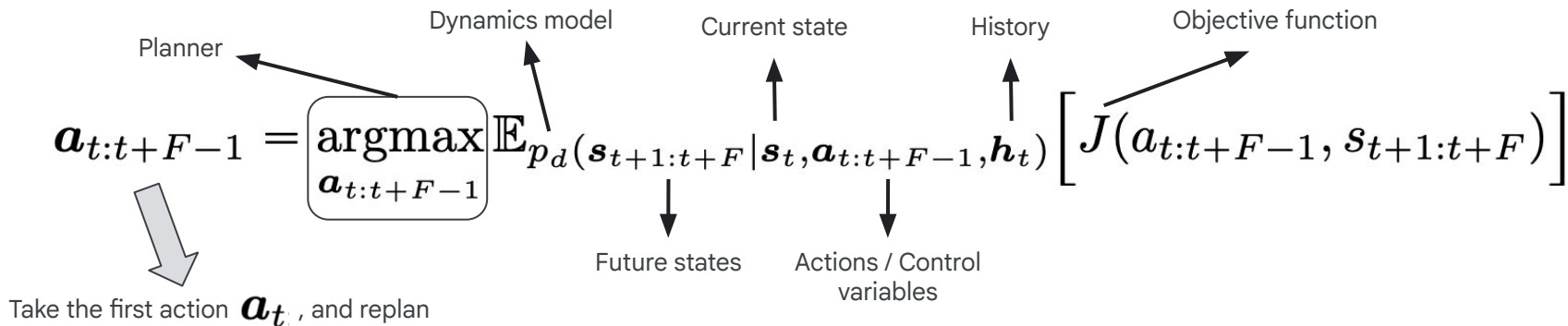
Model Predictive Control (MPC)

Algorithm 1: Main MPC loop.

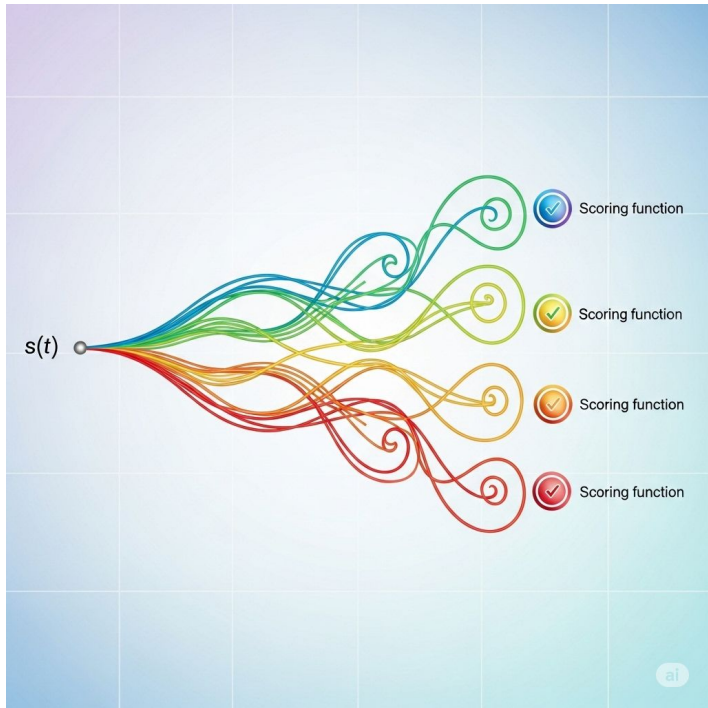
```

1 Input:  $\mathcal{D}$  = offline dataset,  $N$  = num. samples,  $F$  = forecast horizon,  $H$  = history length
2  $(p_d, \rho, J)$  = train( $\mathcal{D}$ )
3  $s_0$  = env.init()
4  $h_0 = (s_0)$ 
5 for  $t = 0 : \infty$  do
6    $a_t$  = planner.plan( $s_t, h_t, p_d, \rho, J, N, F, H$ )
7    $(s_{t+1}, r_{t+1})$  = env.step( $s_t, a_t$ )
8    $h_t$  = append( $a_t, s_{t+1}, r_{t+1}$ )
9    $h_t$  = suffix( $h_t, H$ )

```

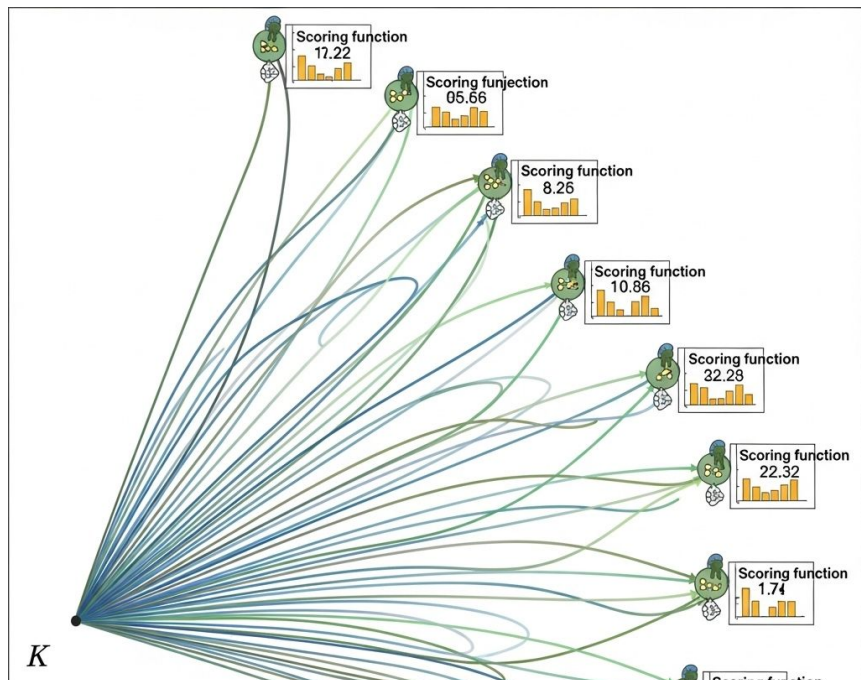


Model Predictive Control (MPC)



$$\mathbf{a}_{t:t+F-1} = \underset{\mathbf{a}_{t:t+F-1}}{\operatorname{argmax}} \mathbb{E}_{p_d(\mathbf{s}_{t+1:t+F} | \mathbf{s}_t, \mathbf{a}_{t:t+F-1}, \mathbf{h}_t)} \left[J(\mathbf{a}_{t:t+F-1}, \mathbf{s}_{t+1:t+F}) \right]$$

Model Predictive Control (MPC)



$$\mathbf{a}_{t:t+F-1} = \underset{\mathbf{a}_{t:t+F-1}}{\operatorname{argmax}} \mathbb{E}_{p_d(\mathbf{s}_{t+1:t+F} | \mathbf{s}_t, \mathbf{a}_{t:t+F-1}, \mathbf{h}_t)} \left[J(\mathbf{a}_{t:t+F-1}, \mathbf{s}_{t+1:t+F}) \right]$$

Diffusion Model Predictive Control (D-MPC)

Diffusion policy Diffusion world model Reward

$$\mathbf{a}_{t:t+F-1} = \overbrace{\operatorname{argmax}}_{\mathbf{a}_{t:t+F-1}} \overbrace{\mathbb{E}_{p_d(\mathbf{s}_{t+1:t+F} | \mathbf{s}_t, \mathbf{a}_{t:t+F-1}, \mathbf{h}_t)}} \left[\overbrace{J(\mathbf{a}_{t:t+F-1}, \mathbf{s}_{t+1:t+F})} \right]$$

- Joint trajectory-level world model suffers less from compounding errors than a standard one-step model when rolling out
- Diffusion captures multimodality of the forecast
- Policy is used as a proposal inside of a sampling based optimizer
- We train the world model and policy proposal using supervised learning with diffusion loss on reward-free "play" data.

MPC: Pros and Cons

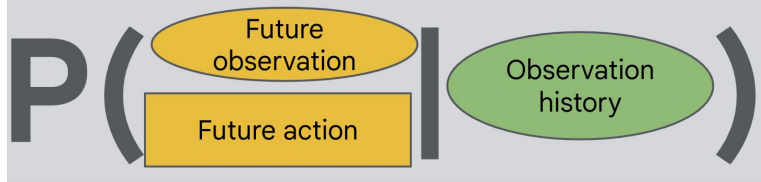
- Advantages:
 - Can adapt to novel reward functions at test time
 - Dynamics models are easier to learn and transfer better than policies
 - Easier adaptation to novel dynamics at test time (e.g., multi-agent settings)
- Disadvantages
 - Slower (needs more test-time compute to sample from policy proposal)

Exploring the design space of diffusion-based agents

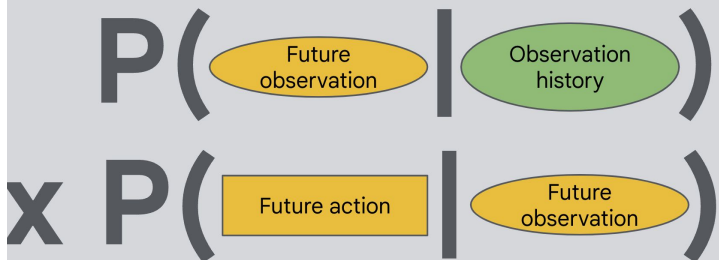
Diffusion policy



Diffuser

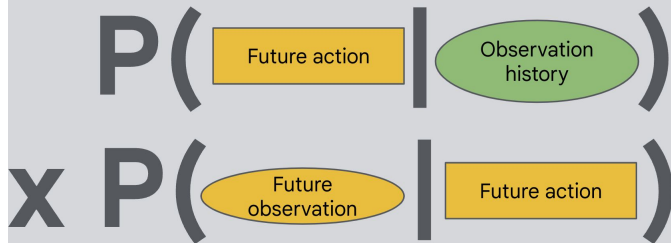


Decision Diffuser



- Diffusion policy: SOTA on complex continuous control
 - But requires expert demonstrations
- Diffuser: joint state-action modeling
 - Implicit world model, model-based planning
- Decision diffuser: observation-only learning
 - Learning from video-only data!
- Diffusion MPC: action proposal + world modeling
 - Runtime adaptation to novel rewards and novel dynamics

Diffusion Model Predictive Control (D-MPC)



D-MPC obtains competitive results in fixed-reward single-task setups

Domain	Level	MOREL	MBOP	D-MPC (ours)	Diffuser	DT	BC	CQL	IQL
halfcheetah	medium	42.10	44.60	46.00 (± 0.17)	44.20	42.60	42.60	44.00	47.40
hopper	medium	95.40	48.80	61.24 (± 2.30)	58.50	67.60	52.90	58.50	66.30
walker2d	medium	77.80	41.00	76.21 (± 2.67)	79.70	74.00	75.30	72.50	78.30
halfcheetah	medium-replay	40.20	42.30	41.12 (± 0.31)	42.20	36.60	36.60	45.50	44.20
hopper	medium-replay	93.60	12.40	92.49 (± 2.23)	96.80	82.70	18.10	95.00	94.70
walker2d	medium-replay	49.80	9.70	78.81 (± 4.19)	61.20	66.60	26.00	77.20	73.90
Locomotion Average		66.48	33.13	65.98	63.77	61.68	41.92	65.45	67.47

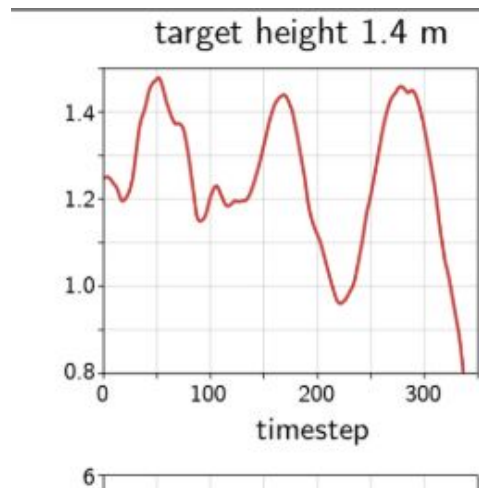
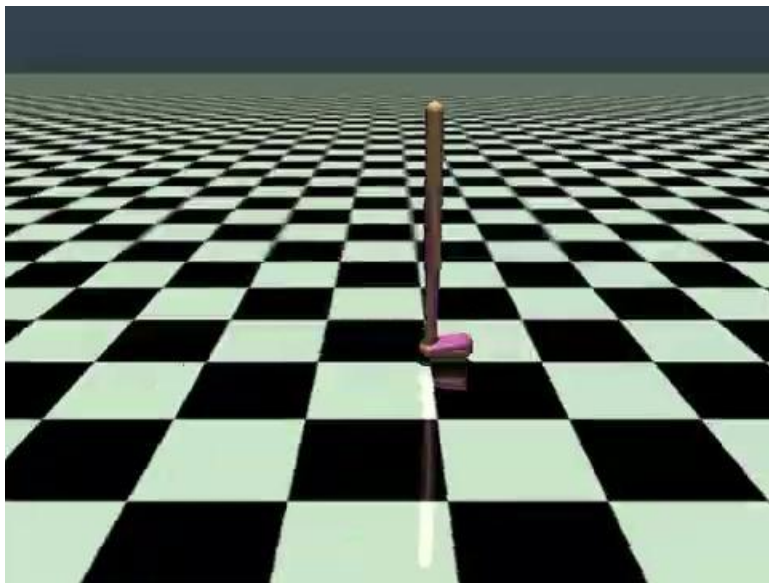
Domain	Level	MBOP	D-MPC (ours)	DT	BC	CQL	IQL
adroit-pen	cloned	63.20	89.22 (± 12.57)	71.17 (± 2.70)	99.14 (± 12.27)	14.74 (± 2.31)	114.05 (± 4.78)
adroit-door	cloned	0.00	16.36 (± 2.20)	11.18 (± 0.96)	3.40 (± 0.95)	-0.08 (± 0.13)	9.02 (± 1.47)
adroit-hammer	cloned	4.20	12.27 (± 3.58)	2.74 (± 0.22)	8.90 (± 4.04)	0.32 (± 0.03)	11.63 (± 1.70)
Adroit Average		22.47	39.28	28.36	37.15	4.99	44.90

Domain	Level	D-MPC (ours)	BC	CQL	IQL
kitchen	mixed	67.50 (± 2.09)	51.50	52.40	51.00
kitchen	partial	73.33 (± 1.64)	38.00	50.10	46.30
Kitchen Average		70.42	44.75	51.25	48.65

Table 2: Performance comparison of D-MPC with various model-based and model-free offline RL methods across different domains. Baseline results are obtained from existing papers (Ajay et al., 2023; Tarasov et al., 2024). Performance is measured using normalized scores (Fu et al., 2020). For D-MPC, we report the mean and standard error of normalized scores over 30 episodes with different random initial environment conditions. Following (Kostrikov et al., 2021), we highlight in bold scores within 5% of the maximum per task. Baseline numbers from (Ajay et al., 2023) do not have associated standard errors. We include the standard errors for baseline numbers from (Tarasov et al., 2024) when they are present.

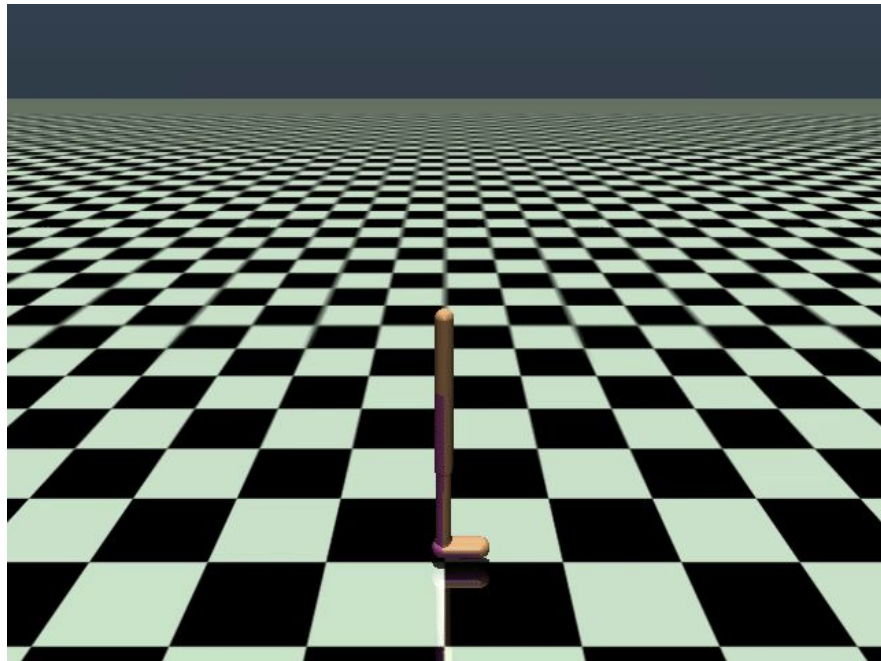
D-MPC can adapt to novel rewards at run time

Note: Initial height of the walker is 1.25 m



D-MPC can adapt to novel dynamics while diffuser struggles

A walker with a broken left ankle



The action proposal / dynamics model factorization in D-MPC allows recovery by fine-tuning the dynamics model with additional “play” data

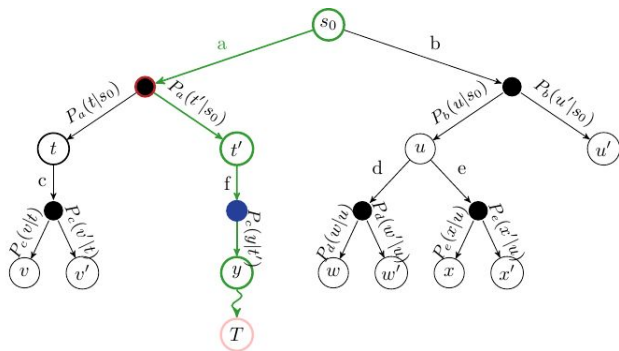
	Diffuser	D-MPC
Original	79.60	76.21 (± 2.67)
Pre-FT w/ defect	25.85 (± 1.08)	22.74(± 1.41)
Post-FT w/ defect	6.8(± 0.86)	30.65(± 1.89)

Performance on Walker2D before and after a simulated hardware defect, followed by fine-tuning (FT) on play data.

Ways to use World Model

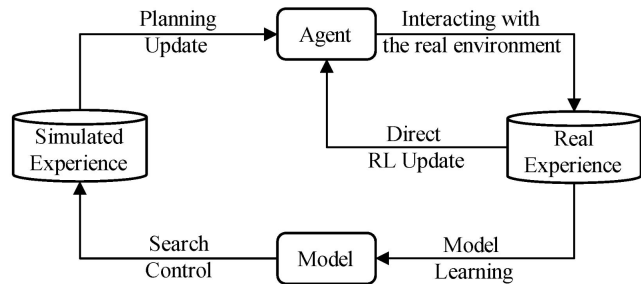
Decision time planning

- + Can do an adaptive amount of planning.
- + Can be used with any new reward fn, e.g. intrinsic reward from LLM.
- + Less sensitive to model error; results in exploration.
- Requires slow planning at test time.



Background planning

- + Planning cost is amortized into a policy.
- + Can improve data efficiency wrt environment interactions.
- A bad model can pollute the policy if the generated trajectories are OOD.
- ? Can be combined with learning from real data.



Relevant paper:

Alver, Safa, and Doina Precup. 2022. "A Look at Value-Based Decision-Time vs. Background Planning Methods across Different Settings." arXiv [Cs.LG]. arXiv. <http://arxiv.org/abs/2206.08442>.

Latent world model for background planning (ICML 2025)

[Submitted on 3 Feb 2025 ([v1](#)), last revised 16 Jul 2025 (this version, v3)]

Improving Transformer World Models for Data-Efficient RL

Antoine Dedieu, Joseph Ortiz, Xinghua Lou, Carter Wendelken, Wolfgang Lehrach, J Swaroop Guntupalli, Miguel Lazaro-Gredilla, Kevin Patrick Murphy

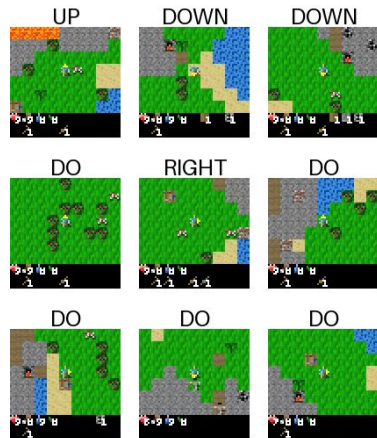
Primary test bed: Craftax

Current testbed: [Craftax-classic](#)

- JAX version of [Crafter](#), which is a 2D version of Minecraft.
- Run 250k steps/s on a single GPU!
- Sparse reward, partially observability, stochastic dynamics, procedurally generated.

Goal:

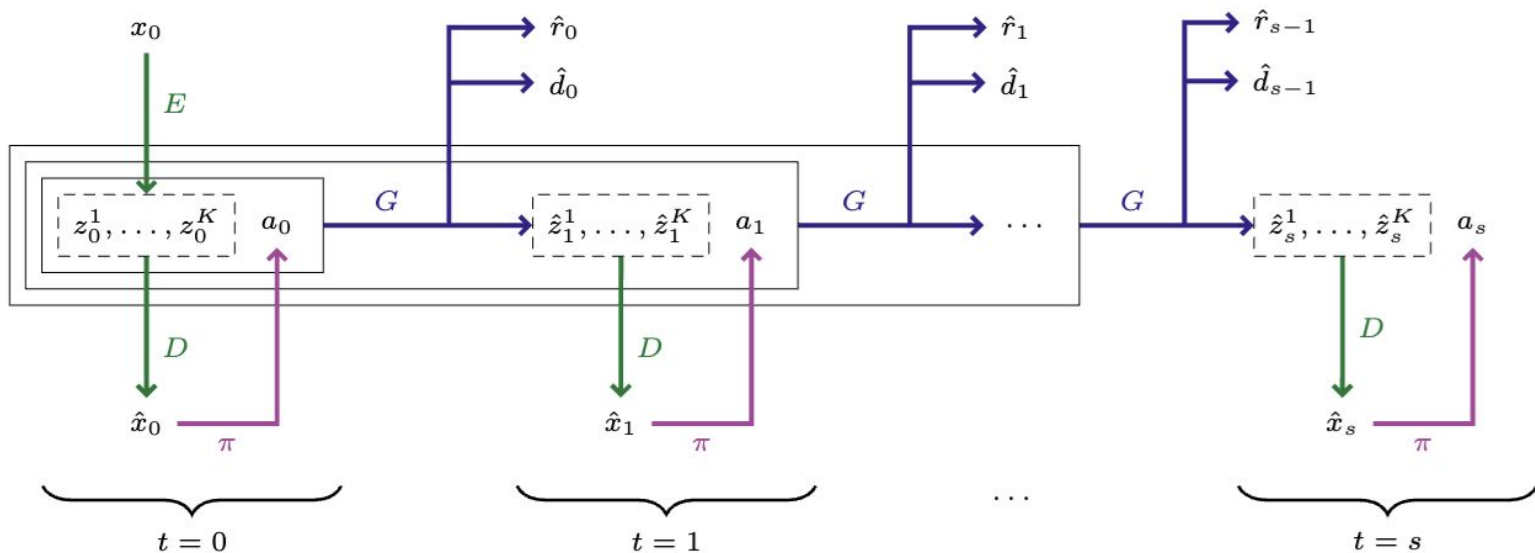
Can we design a new SOTA approach on Craftax-classic when only interacting with the environments 1M steps?



MBRL baseline: Train MFRL policy inside Transformer WM

We model the dynamics using a transformer WM as in [IRIS](#):

- A VQ-VAE trained with **l2 loss**, to map each image into 64 discrete latent codes
- TWM predicts future code indices z_{t+1}^k over time given history of code indices and actions
- TWM also predicts reward and terminal states
- We train the policy using imaginary rollouts



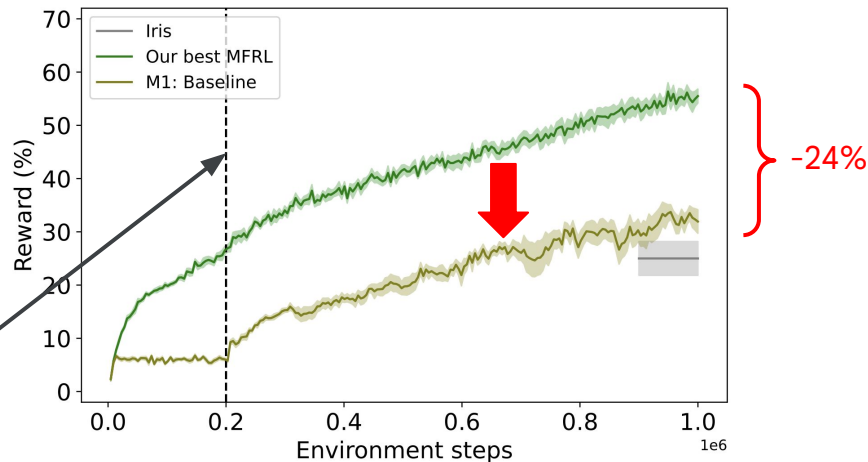
MBRL baseline is worse than MFRL baseline!

Limitation: despite combining SOTA progress in MFRL and generative modeling, the MBRL baseline agent, **only trained in imagination**, is largely outperformed by our best MFRL agent

Solution:

We propose 3 simple improvements that progressively converts this baseline into a SOTA agent

Start training in
imagination after
200k steps



MBRL improvement 1: Dyna with warmup

Limitation:

Most SOTA MBRL agents (IRIS, DreamerV3) are only trained on imaginary data

Solution:

Train policy on real and imaginary data (after WM is sufficiently well-trained)

Without Steps 3 and 4, this is MFRL

Algorithm 1 MBRL agent. See Appendix A.3 for details.

Input: number of environments N_{env} ,
environment dynamics $\mathcal{M}_{\text{env}}$,
rollout horizon for environment T_{env} and for TWM T_{WM} ,
background planning starting step T_{BP} ,
total number of environment steps T_{total} ,

Initialize: observations $O_1^n \sim \mathcal{M}_{\text{env}}$ for $n = 1 : N_{\text{env}}$,
data buffer $\mathcal{D} = \emptyset$,
TWM model $\mathcal{M}$ and parameters Θ ,
AC model π and parameters Φ ,
number of environment steps $t = 0$.

repeat

// 1. Collect data from environment

$\tau_{\text{env}}^n = \text{rollout}(O_1^n, \pi_\Phi, T_{\text{env}}, \mathcal{M}_{\text{env}})$, $n = 1 : N_{\text{env}}$

$\mathcal{D} = \mathcal{D} \cup \tau_{\text{env}}^{1:N}$; $O_1^{1:N} = \tau_{\text{env}}^{1:N}[-1]$; $t+ = N_{\text{env}}T$

// 2. Update policy on environment data

$\Phi = \text{PPO-update-policy}(\Phi, \tau_{\text{env}})$

// 3. Update world model

$\tau_{\text{replay}}^n = \text{sample-trajectory}(\mathcal{D}, T_{\text{WM}})$, $n = 1 : N_{\text{env}}$

$\Theta = \text{update-world-model}(\Theta, \tau_{\text{replay}}^{1:N_{\text{env}}})$

// 4. Update policy on imagined data

if $t \geq T_{\text{BP}}$ **then**

$O_1^n = \text{sample-obs}(\mathcal{D})$, $n = 1 : N_{\text{env}}$

$\tau_{\text{WM}}^n = \text{rollout}(O_1^n, \pi_\Phi, T_{\text{WM}}, \mathcal{M}_\Theta)$, $n = 1 : N_{\text{env}}$

$\Phi = \text{PPO-update-policy}(\Phi, \tau_{\text{WM}}^{1:N_{\text{env}}})$

end if

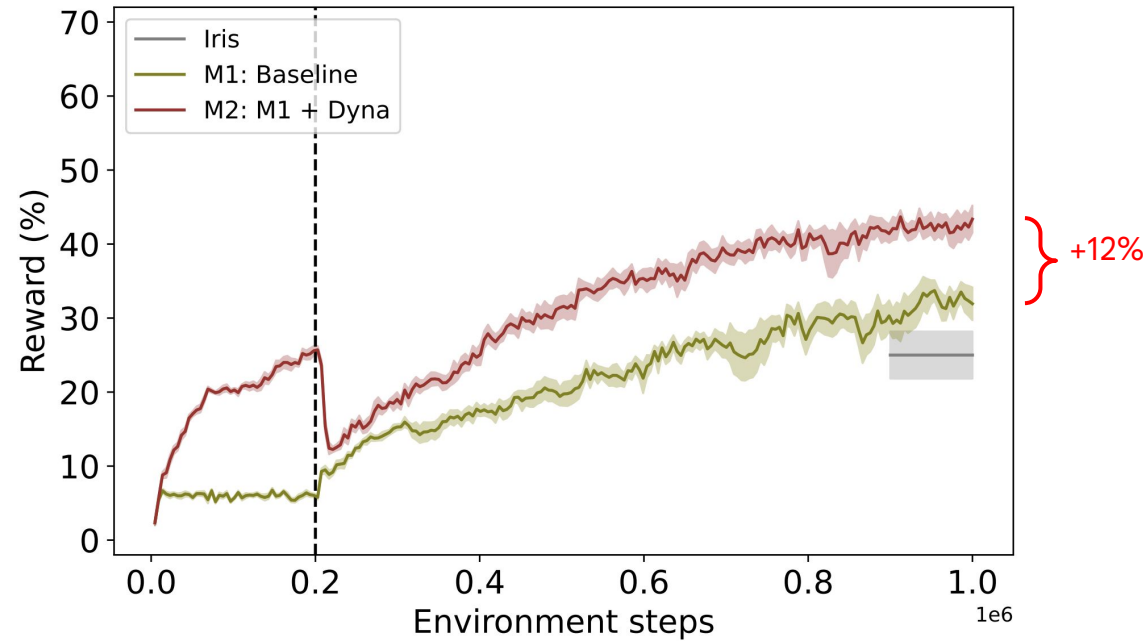
until $t \geq T_{\text{total}}$

Solution:

Perform on-policy updates on real and imaginary data

Wait for the WM to be sufficiently accurate so that it does not “pollute” the replay buffer

MBRL improvement 1: Performance gain



MBRL improvement 2: Patch factorization

Limitation:

VQ-VAE learns codewords that do not correspond to "meaningful entities" in the environment

Solution:

Exploit domain-specific knowledge to segment image into patches, to ensure a "disentangled" latent representation



Each image O in Craftax-classic is composed of 9×9 patches, of size 7×7 each

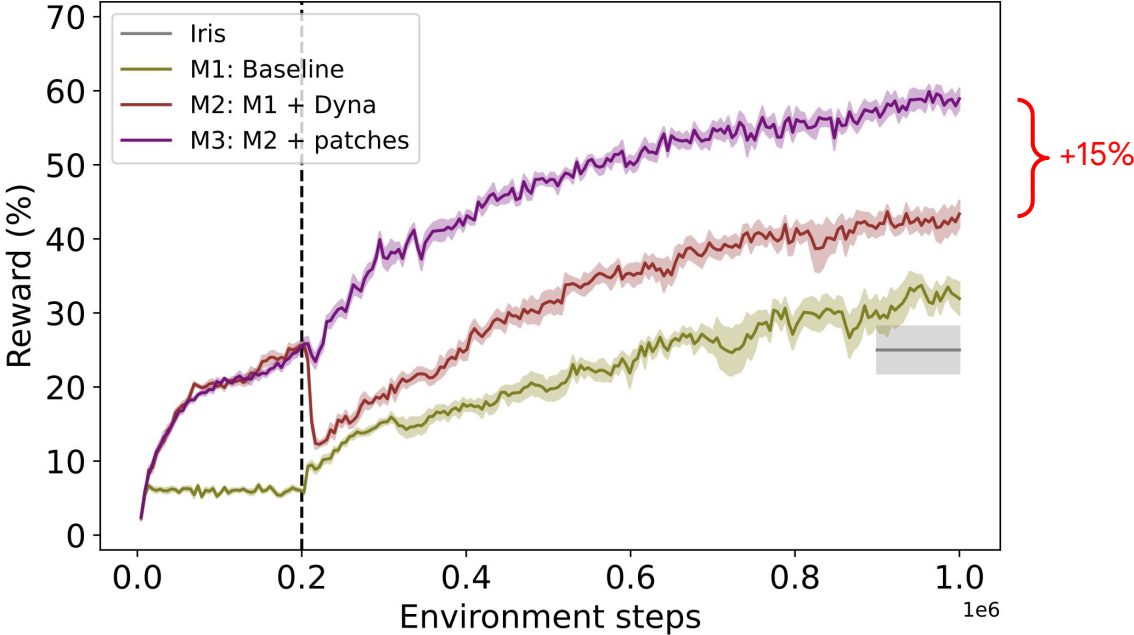
$$O = (p^1, \dots, p^L)$$

We factorize the encoder and independently quantize each patch, returning a sequence of indices

$$(q^1, \dots, q^L) = (\text{enc}(p^1), \dots, \text{enc}(p^L))$$

Similarly, the decoder decodes each token into a patch.

MBRL improvement 2: Performance gain



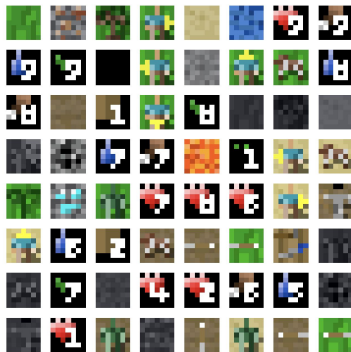
MBRL improvement 2': Nearest neighbor tokenizer

Limitation: VQ-VAE learns codes that evolves through training, which makes TWM training hard

Solution: we introduce a nearest neighbor tokenizer (NNT) which, for a patch p and a current codebook $\mathcal{C}_{\text{NN}} = \{e_1, \dots, e_K\}$ adds p to the codebook if it is farther away than a threshold τ

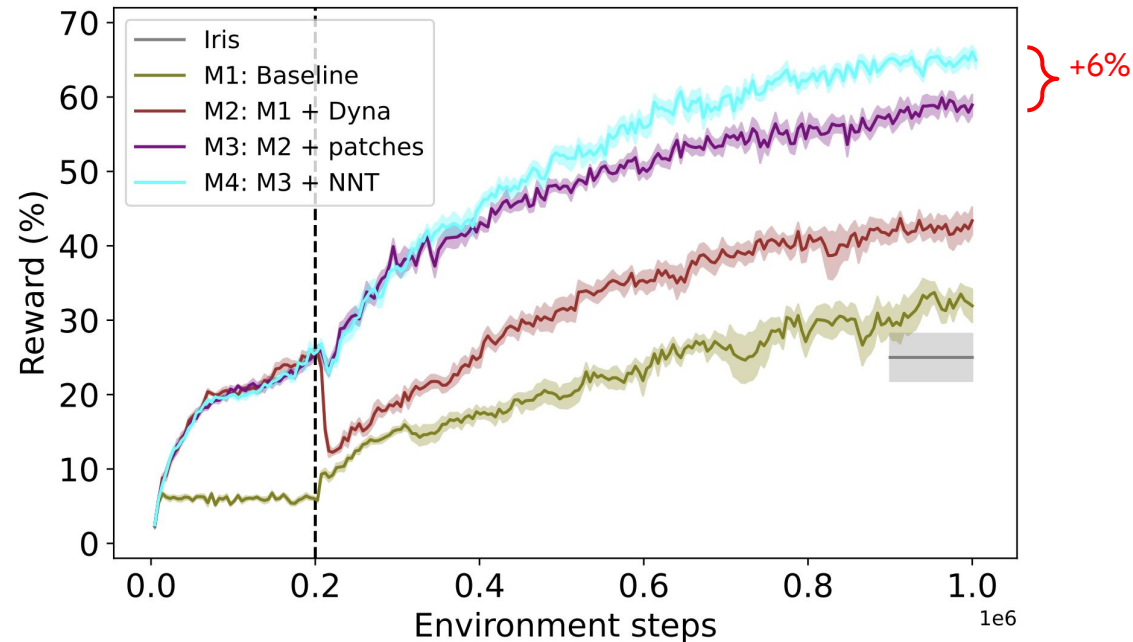
$$q = \text{enc}(p) = \begin{cases} \underset{1 \leq i \leq K}{\text{argmin}} \|p - e_i\|_2^2 & \text{if } \min_{1 \leq i \leq K} \|p - e_i\|_2^2 \leq \tau \\ K + 1 & \text{otherwise.} \end{cases}$$

The codes added with NNT are never updated: having a fixed target makes TWM training easier



Example of 64 codes learned with NNT

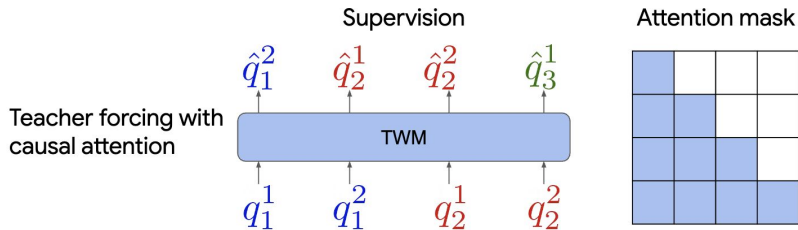
MBRL improvement 2': Performance gain



MBRL improvement 3: Block teacher forcing

Limitation: TWMs are trained to generate tokens autoregressively over time and within a timeslice, which imposes an unnatural ordering of tokens

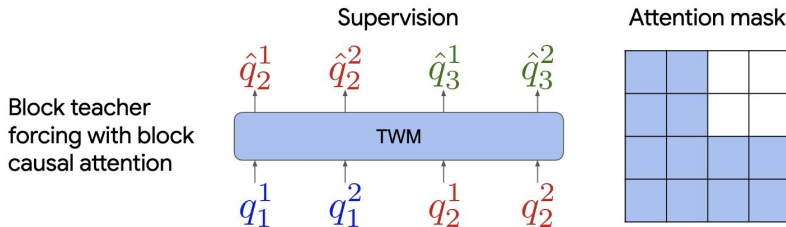
$$\mathcal{L}_{\text{TF}} = \log \prod_{t=1}^T \prod_{i=1}^L \mathcal{L}_t^i, \quad \mathcal{L}_t^i = p(q_{t+1}^i | q_{1:t}^{1:L}, q_{t+1}^{1:i-1}, a_{1:t})$$



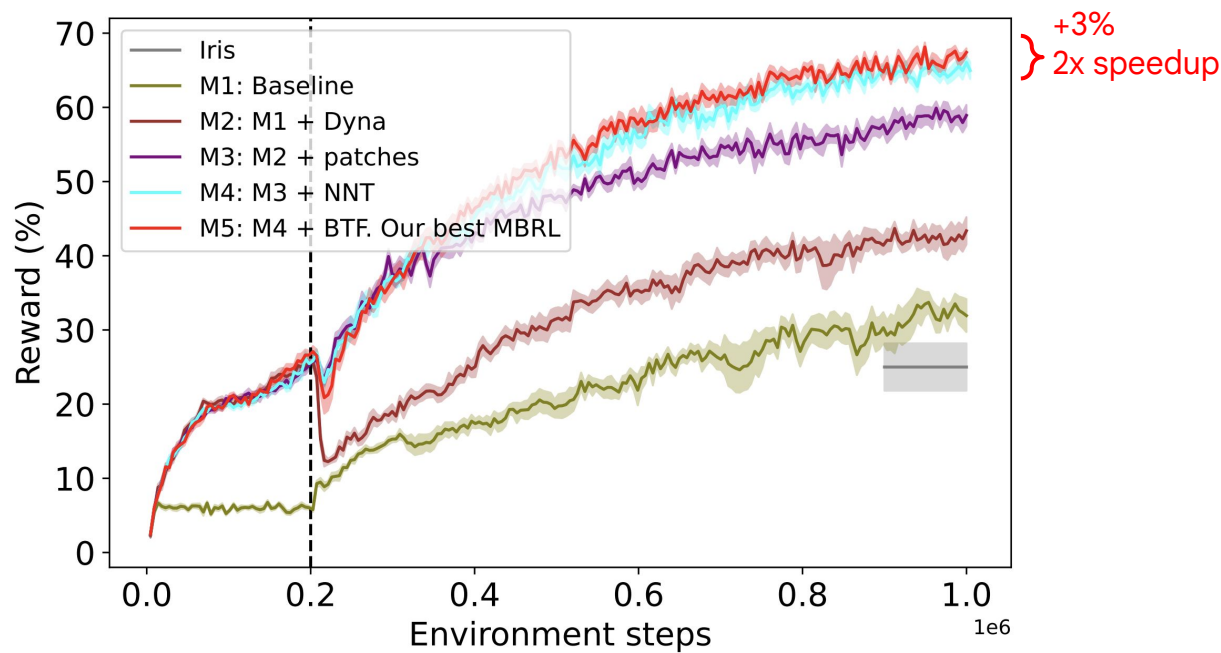
Solution: we propose block teacher forcing which:

- allows the model to reason jointly about the future tokens of the next timestep before sampling them in parallel.
- introduces a block causal attention mask
- modifies the supervision of the TWM

$$\mathcal{L}_{\text{BTF}} = \log \prod_{t=1}^T \prod_{i=1}^L \tilde{\mathcal{L}}_t^i, \quad \tilde{\mathcal{L}}_t^i = p(q_{t+1}^i | q_{1:t}^{1:L}, a_{1:t})$$



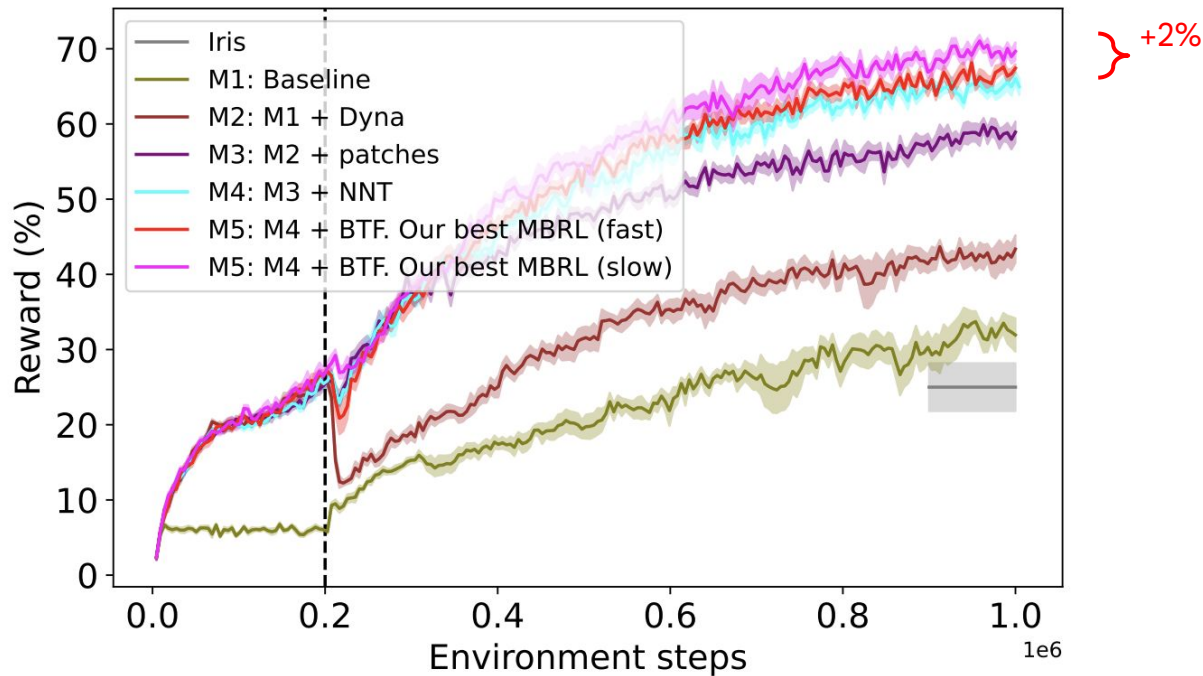
MBRL improvement 3: Performance gain



Training TWM longer

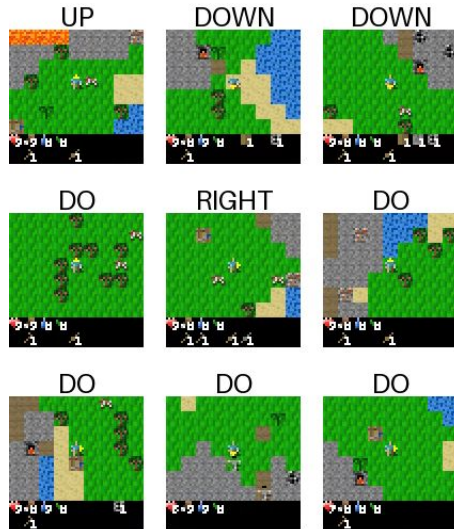
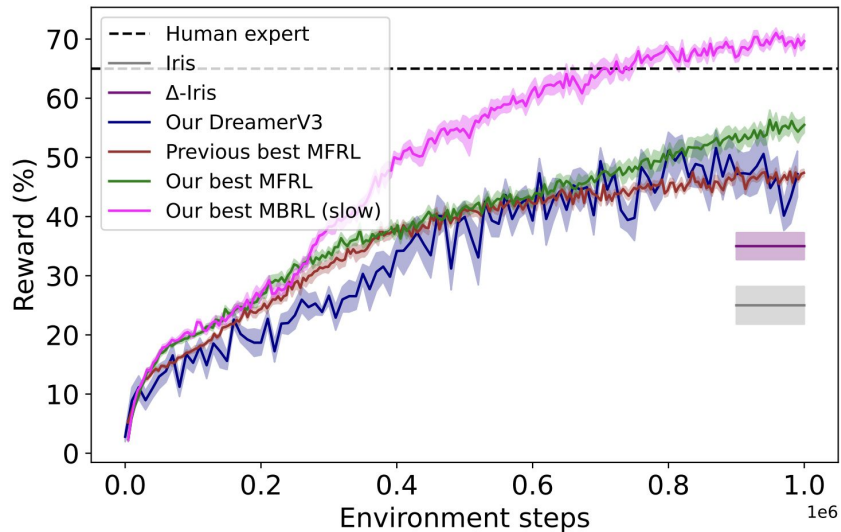
Fast version: 500 train steps, 8h training.

Slow version: 4k train steps, 2 days training.

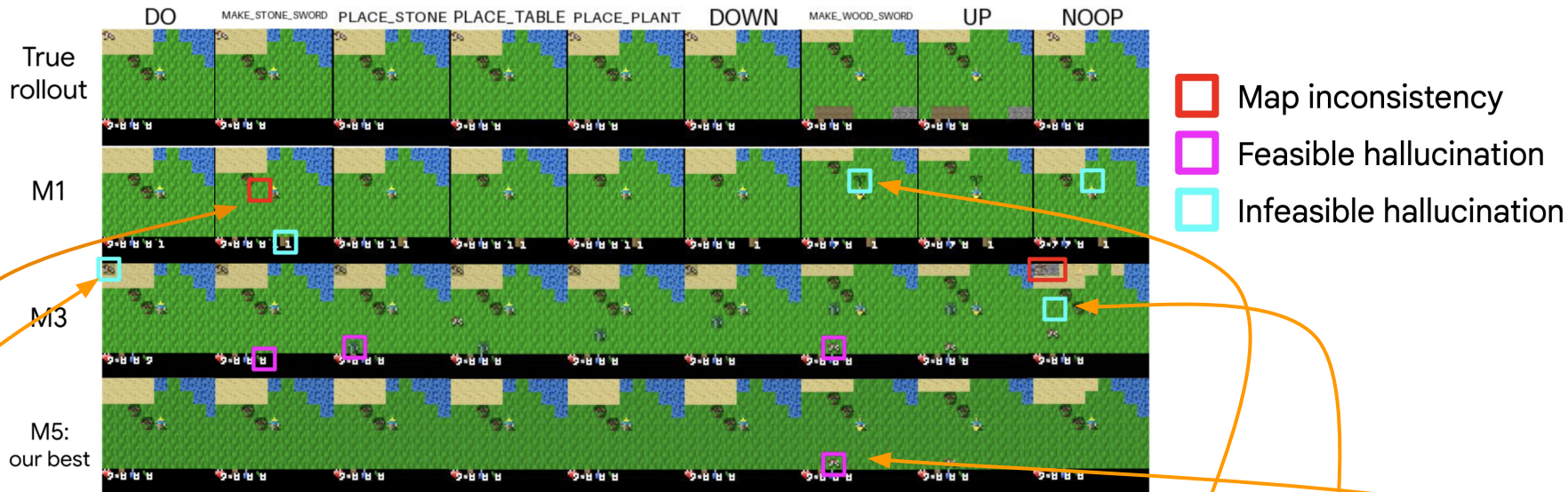


Putting it all together to achieve superhuman reward

After climbing the ladder, our best MBRL agent achieves, for the first time, superhuman reward on Craftax-Classic, outperforming all the previous MFRL and MBRL agents (DreamerV3, IRIS...)



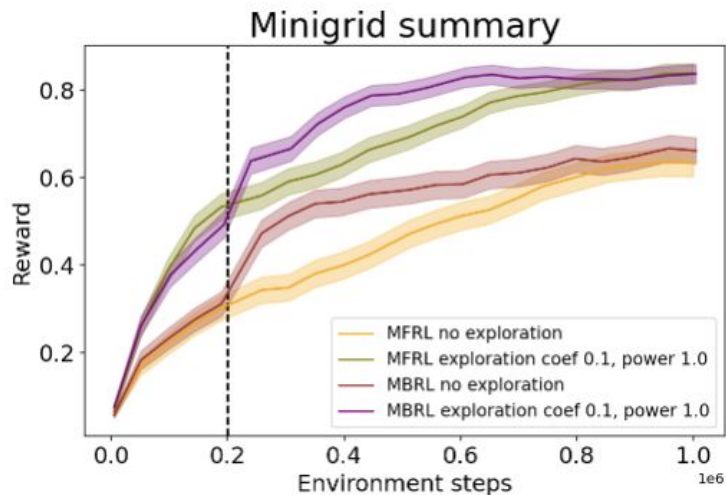
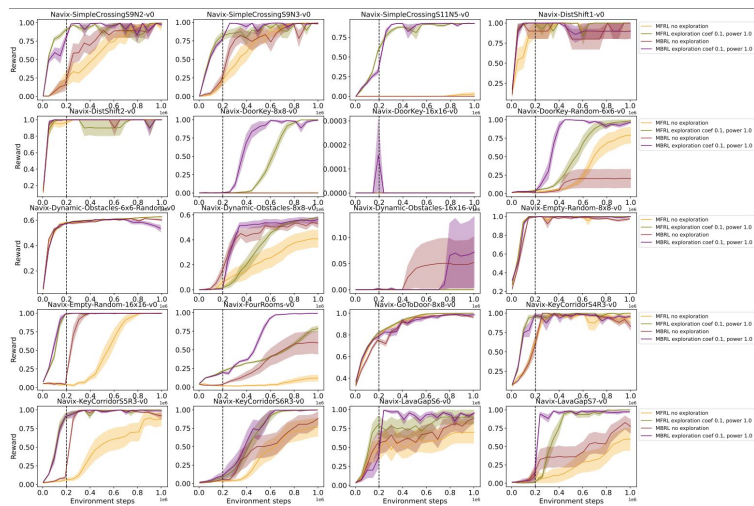
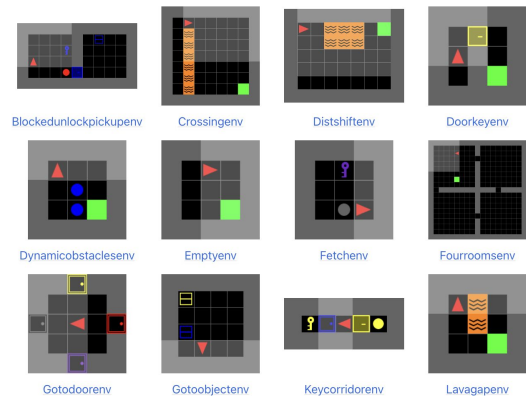
Comparing TWM rollouts: qualitative evaluations



- M1 makes simple map errors in successive timesteps + generates hallucinations which do not respect the game dynamics: adding wood to inventory + placing plants
- M3 generates some infeasible hallucinations: cow in wrong place + alien disappears
- M5 generates feasible hallucinations (cow)

Minigrid

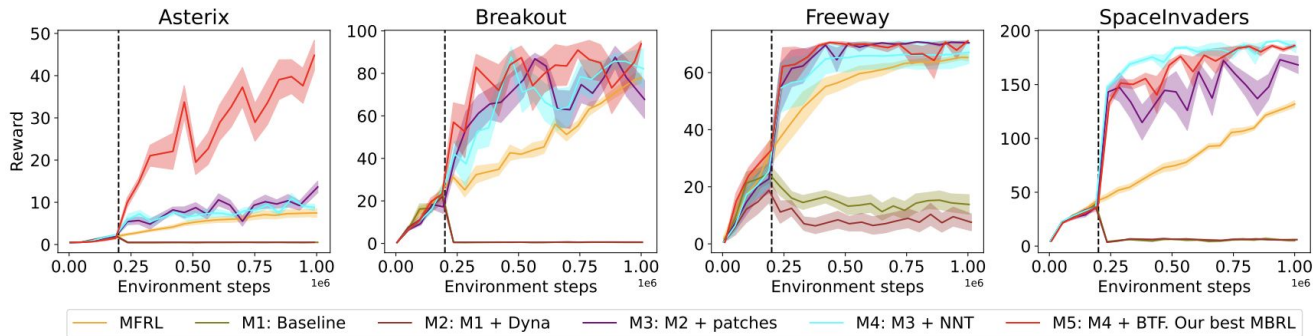
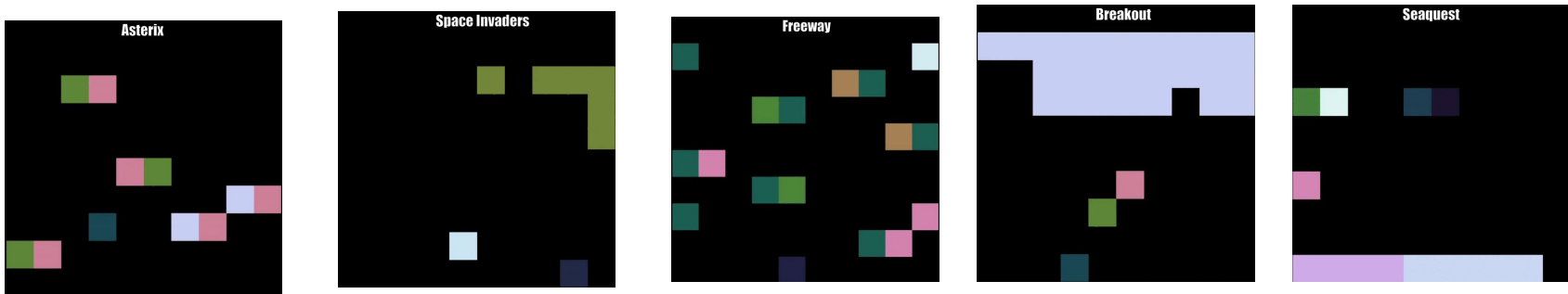
Using 20 environments from Navix (JAX-native Minigrid)
Simple exploration bonus in collected and imaginary rollouts



Mini Atari

MinAtar:

- 5 Simplified Atari games
- 10x10xK observations, binary rewards

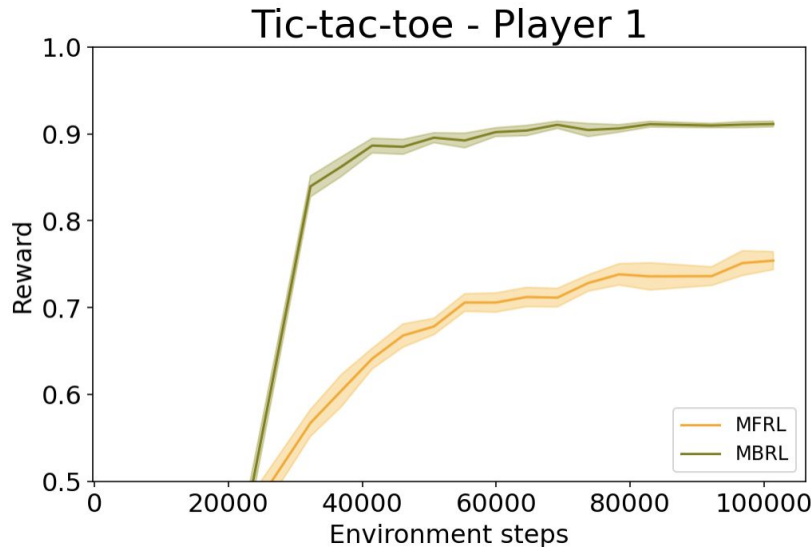
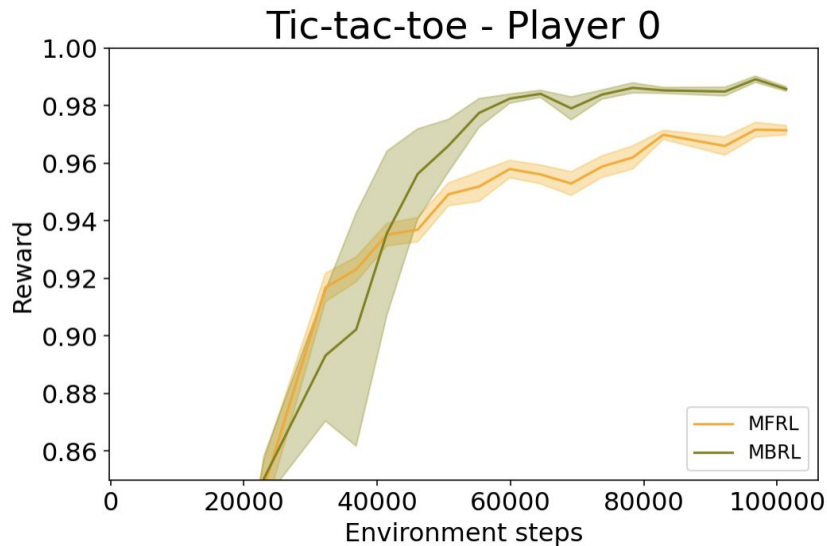


tic-tac-toe

MBRL pipeline adapted to model the sequence of observations and actions coming from both players

$x_t^{*1}, a_t^{*1}, x_t^{*2}, 1, a_t^{*2}, x_{t+1}^{*1}$

- TWM now has heads to predict `player_id` and `legal_action_move`
- Can generate rollouts which respect game dynamics
- Performance against a random player



Summary

- Generative AI
 - "MUSE"
 - "Distributional diffusion" (images)
- Agentic AI
 - "Distributional diffusion" (robot actions)
 - "Diffusion Model Predictive Control"
 - "Improving Transformer World Models for Data-Efficient RL"

Ongoing / future work

- World models that don't predict/reconstruct all the pixels (cf JEPA)
 - Pixel generation is overkill for many agentic tasks
 - Ongoing work to predict the future trajectory (in latent space) of point tracks rather than pixels (for robotics tasks)
- World models that can be learned from much less data
 - Using LLMs to generate code world models (in-context learning)
- Beyond single-step world models
 - Learning temporally abstract world models for efficient planning

Ongoing / future work

- World models that can be learned from much less data
 - Using LLMs to generate code world models (in-context learning)



Ongoing / future work

- World models that don't predict/reconstruct all the pixels (cf JEPA)



David Pfau
@pfau



This is really impressive work, and congrats to the team, but as an aside...does anyone else find it weird that "world model" evolved from meaning "the minimal model needed to plan in an environment" to "action-conditional video model"?



Google DeepMind 🏆 @GoogleDeepMind · Aug 5

What if you could not only watch a generated video, but explore it too? 🌐

Genie 3 is our groundbreaking world model that creates interactive, playable environments from a single text prompt.

...



8:17 AM · Aug 5, 2025 · 93.5K Views

David Pfau @pfau · Aug 5



Like, this is what world models were supposed to be. It was explicitly *not* about accurately rendering every pixel, it was about throwing out any information not needed for acting optimally in the world.



Ongoing / future work

- Beyond single-step world models
 - Learning temporally abstract world models for efficient planning



Richard Sutton ✓

@RichardSSutton



The one-step trap (in AI research)

The one-step trap is the common mistake of thinking that all or most of an AI agent's learned predictions can be one-step ones, with all longer-term predictions generated as needed by iterating the one-step predictions. The most important place where the trap arises is when the

Last edited 10:21 AM · Jul 18, 2024 · **58.9K** Views